Machine Learning Techniques (BCS-055)

Unit 1: Introduction & Learning Systems

Q1. Short Definitions (2 Marks Each)

(a) Define a well-posed learning problem.

- **Tom Mitchell's definition:** *"A computer program is said to **learn from experience E** with respect to some **class of tasks T** and **performance measure P**, if its performance at tasks in T, as measured by P, **improves with experience E**."*
- A learning problem is **well-posed** when **T, P and E are all clearly specified**.
- *Example (Checkers):* **T** = playing checkers, **P** = percentage of games won against opponents, **E** = playing practice games against itself.

(b) Differentiate between Machine Learning and Deep Learning.

Basis	Machine Learning (ML)	Deep Learning (DL)
Definition	Algorithms that learn patterns from data	A subset of ML using **multi-layer neural networks**
Feature extraction	**Manual** (by domain experts)	**Automatic** (learned by the network)
Data requirement	Works with **small or medium** data	Needs **large** datasets
Hardware	Runs on a CPU	Needs a **GPU or TPU**
Training time	Short	Long
Interpretability	High (for example decision trees)	Low ("black box")
Examples	SVM, Decision Tree, k-NN, Naive Bayes	CNN, RNN, LSTM, Transformers

(c) What is Occam's Razor in ML?

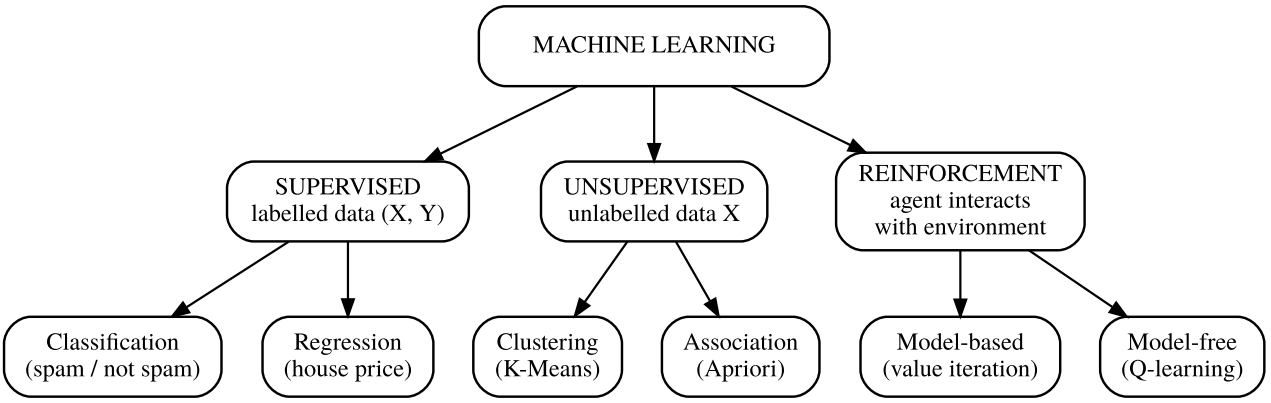
- **Occam's Razor** says: *"Prefer the **simplest hypothesis** that fits the data."*
- Among several models with similar training accuracy, choose the one with the **fewest assumptions or parameters**, because simpler models **generalize better** and are **less likely to overfit**.
- *Example:* ID3 prefers **shorter decision trees** over longer ones. Regularization (L1/L2) penalizes complex models.

Q2. Compare supervised, unsupervised and reinforcement learning with real-life examples. (7 Marks)

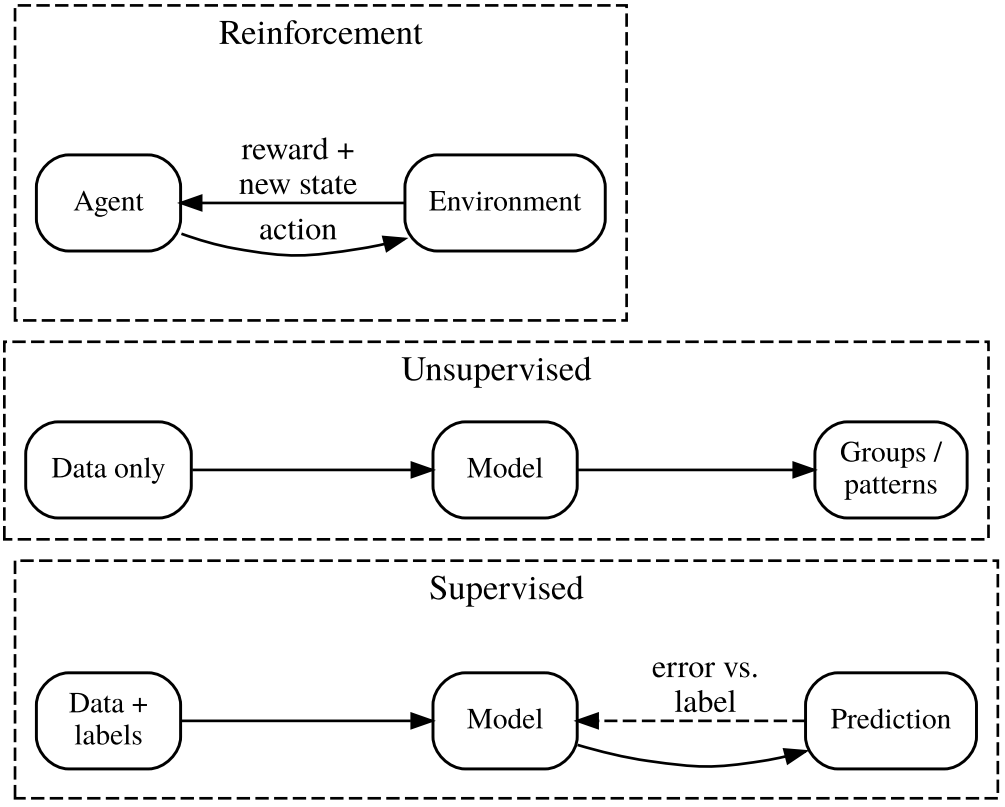
1. Introduction / Definition

Machine learning is divided into three main paradigms according to the **type of feedback** available. **Supervised learning** learns from **labelled data** (input-output pairs). **Unsupervised learning** finds **hidden patterns in unlabelled data**. **Reinforcement learning** learns by **interacting with an environment** and receiving **rewards or penalties**.

2. Block Diagram



The three learning paradigms



How each paradigm learns

3. Core Explanation / Working

(a) Supervised Learning

- The model is trained on a **labelled dataset**, in which every input x has a known output y. It learns a mapping **f: X → Y**.
- Training **minimizes the error** between the predicted and actual outputs.
- **Types:**
  - **Classification:** the output is a category. Example: email spam or not spam.
  - **Regression:** the output is a continuous value. Example: house price prediction.
- **Algorithms:** Linear/Logistic Regression, Decision Trees, SVM, k-NN, Naive Bayes.
- **Real-life examples:** spam filtering in Gmail, disease diagnosis from X-rays, credit-score prediction.

(b) Unsupervised Learning

- The model gets **only inputs, with no labels**, and must discover **structure** on its own.
- **Types:**
  - **Clustering:** groups similar items together.
  - **Association:** finds rules like "people who buy bread also buy butter".
  - **Dimensionality reduction:** PCA.
- **Algorithms:** K-Means, Hierarchical Clustering, DBSCAN, Apriori, PCA.
- **Real-life examples: customer segmentation** in marketing, **market-basket analysis** in supermarkets, anomaly (fraud) detection.

(c) Reinforcement Learning

- An **agent** takes **actions** in an **environment**, receives a **reward** (feedback), and learns a **policy** that **maximizes cumulative reward**.
- It learns by **trial and error**. The feedback is **delayed** and not given for each action.
- It must balance **exploration** (trying new actions) with **exploitation** (using the best known action).
- **Algorithms:** Q-Learning, SARSA, Deep Q-Networks (DQN), Policy Gradients.
- **Real-life examples: AlphaGo** (game playing), **self-driving cars**, robot navigation, recommendation systems.

4. Key Characteristics - Comparison Table

Basis	Supervised	Unsupervised	Reinforcement
Input data	Labelled (X, Y)	Unlabelled (X)	No fixed dataset; states and rewards
Feedback	**Direct** (correct answer given)	**None**	**Delayed reward / penalty**
Goal	Predict outputs for new inputs	Discover hidden structure	Learn the best policy (actions)
Problem types	Classification, Regression	Clustering, Association	Control, Decision-making
Algorithms	SVM, Decision Tree, Linear Regression	K-Means, Apriori, PCA	Q-Learning, SARSA, DQN
Human supervision	High	Low	Low (only the reward design)
Real-life example	Spam detection	Customer segmentation	Self-driving car, AlphaGo

5. Advantages & Disadvantages

Supervised: **high accuracy** and easy evaluation, *but* needs **costly labelled data**. *Unsupervised:* **no labelling needed** and finds unknown patterns, *but* results are **hard to evaluate** and interpret. *Reinforcement:* learns **complex sequential behaviour** without a dataset, *but* needs **many trials** (slow training) and careful **reward design**.

6. Conclusion / Application

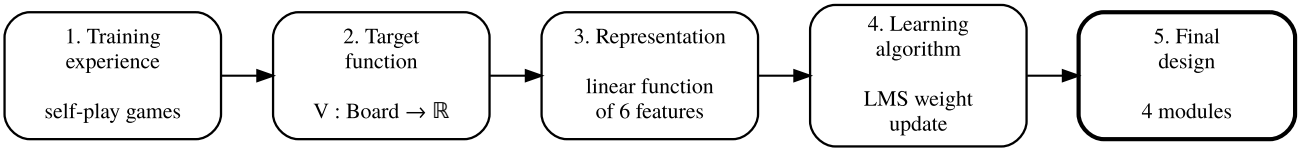
Real systems often combine all three. For example, **ChatGPT** uses supervised pre-training, unsupervised (self-supervised) learning on text, and **reinforcement learning from human feedback (RLHF)**.

Q3. Explain the steps to design a learning system using the Checkers learning program as a case study. (7 Marks)

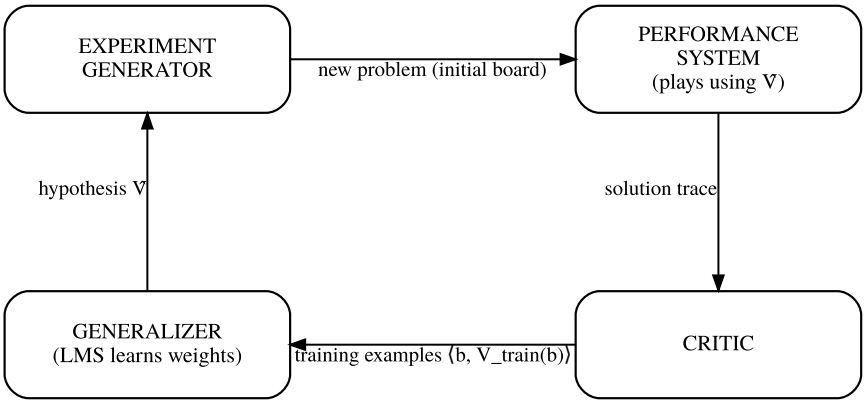
1. Introduction / Definition

Designing a learning system means choosing **how the program will gain experience, what exactly it will learn, how that knowledge is represented, and which algorithm will learn it**. Tom Mitchell explains these design steps using a program that **learns to play checkers**. Its well-posed problem is: **T** = play checkers, **P** = percentage of games won, **E** = games played against itself.

2. Block Diagram - Design Choices and Final Design



Design choices for the checkers learner (the option chosen at each step)



Final design: four modules working in a loop

3. Core Explanation / Working

Step 1 - Choosing the Training Experience

- **Direct or indirect feedback:** direct feedback gives individual board states with the correct move. Indirect feedback gives only the **final result** (win or loss) of a game, which raises the **credit assignment problem**.
- **Degree of control:** the learner can depend on a teacher, or it can **propose its own board states**.
- **Distribution of examples:** the training games should resemble the games that will be tested.

- **Choice:** the program **plays games against itself**, which gives unlimited training data.

Step 2 - Choosing the Target Function

- `ChooseMove: B → M` (board to best move) is hard to learn directly.
- Instead, learn an **evaluation function V: B → R**, which gives a **numeric score to every board state**:
  - V(b) = **+100** if b is a final board state that is **won**.
  - V(b) = **−100** if b is a final board state that is **lost**.
  - V(b) = **0** if b is a final board state that is **drawn**.
  - Otherwise, V(b) = V(b'), where b' is the best final state reachable from b with optimal play. This ideal definition **cannot be computed efficiently**, so we learn an **approximation V̂**.

Step 3 - Choosing a Representation

- V̂ is represented as a **linear combination of 6 board features**:

$$\widehat{V}(b) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + w_5 x_5 + w_6 x_6$$

Feature	Meaning
x1	Number of black pieces on the board
x2	Number of red pieces
x3	Number of black kings
x4	Number of red kings
x5	Number of black pieces threatened by red (can be captured next turn)
x6	Number of red pieces threatened by black

- **Learning V̂ is now reduced to learning the weights w0 to w6.**

Step 4 - Choosing a Function Approximation Algorithm

- **Estimating training values:** only the final outcome is known, so intermediate board values are estimated as `Vtrain(b) ← V̂(Successor(b))` where Successor(b) is the next board state at which it is the program's turn to move again.
- **Adjusting the weights: the LMS (Least Mean Squares) rule.** For each training example `<b, Vtrain(b)>`:
  1. Compute the error: `error(b) = Vtrain(b) − V̂(b)`
  2. Update each weight: **`wi ← wi + η · (Vtrain(b) − V̂(b)) · xi`**
  - η is a small **learning rate** (for example 0.1).
- LMS minimizes the squared error `E = Σ (Vtrain(b) − V̂(b))²`.

Step 5 - The Final Design (four modules)

1. **Performance System:** plays a new game using the current V̂, and outputs the **game history (trace)**.
2. **Critic:** takes the trace and produces **training examples** `<b, Vtrain(b)>`.
3. **Generalizer:** uses LMS on the training examples to produce a **new hypothesis V̂** (new weights).
4. **Experiment Generator:** takes the current hypothesis and proposes a **new problem** (an initial board) to explore.

4. Key Characteristics - Design Choices Summary

Design Step	Options Considered	Choice in Checkers
Training experience	Expert games, self-play, teacher	**Self-play** (indirect feedback)
Target function	ChooseMove: B → M, or V: B → R	**V: B → R**
Representation	Table, rules, polynomial, ANN	**Linear function of 6 features**
Learning algorithm	Gradient descent, LMS, etc.	**LMS weight update**
Training value estimate	Final outcome only	**Vtrain(b) ← V̂(Successor(b))**

5. Advantages & Disadvantages

Advantages

- Gives a **clear, systematic framework** that applies to any learning problem.
- Self-play gives **unlimited training data** at no cost.
- The linear representation is **simple and fast** to learn with LMS.

Disadvantages

- A **linear function of 6 features** may be too simple to capture expert-level strategy.
- **Credit assignment** is hard, because a win or loss gives indirect feedback for every move in the game.

6. Conclusion / Application

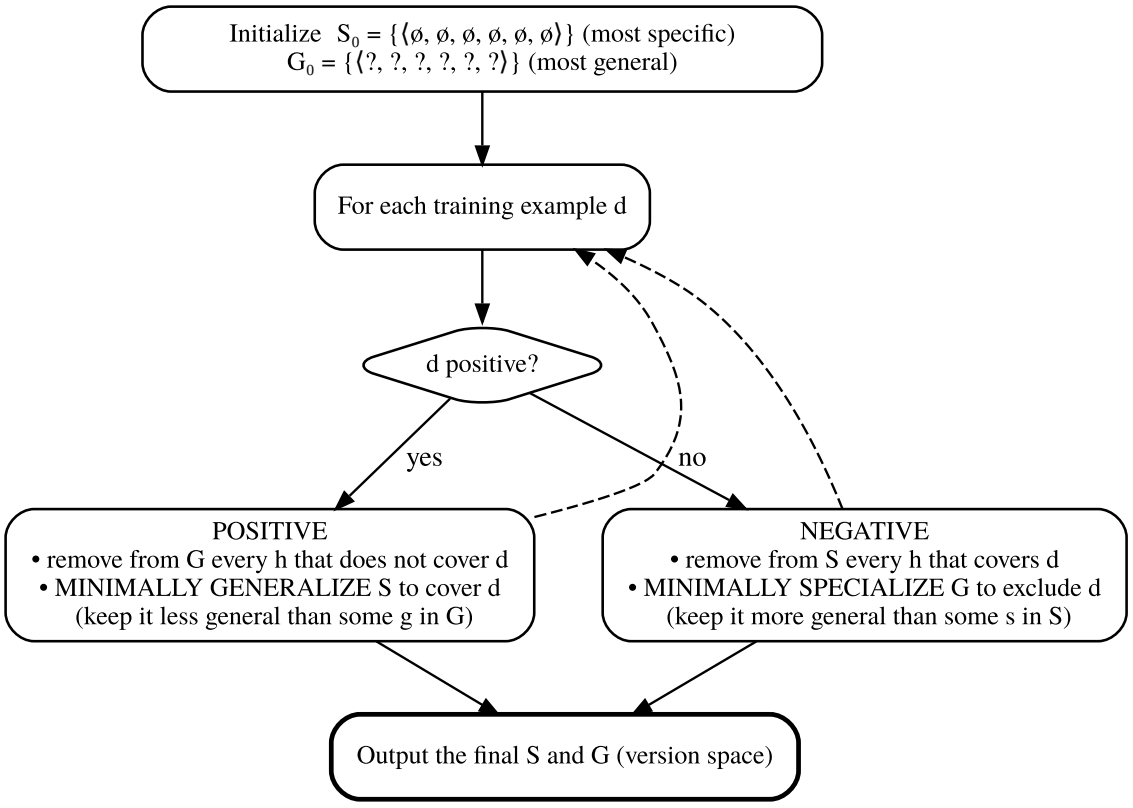
This design framework led to **Samuel's checkers player and TD-Gammon**, and it is the conceptual basis of **AlphaZero**, which learns chess and Go by self-play.

Q4. Trace the Candidate Elimination algorithm to find the maximally general and specific hypotheses. (7 Marks)

1. Introduction / Definition

The **Candidate Elimination (CE) algorithm** finds the **version space**: the set of **all hypotheses consistent with the training data**. It keeps two boundary sets. **S** (specific boundary) holds the **most specific** consistent hypotheses, and **G** (general boundary) holds the **most general** ones. **Positive examples generalize S**, and **negative examples specialize G**.

2. Flowchart - Algorithm



Candidate Elimination algorithm

3. Core Explanation / Working

Training data: EnjoySport (Mitchell)

Ex.	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	**Yes**
2	Sunny	Warm	High	Strong	Warm	Same	**Yes**
3	Rainy	Cold	High	Strong	Warm	Change	**No**
4	Sunny	Warm	High	Strong	Cool	Change	**Yes**

Notation: ? = any value is acceptable, ø = no value is acceptable.

Initialization

- S0 = { <ø, ø, ø, ø, ø, ø> }
- G0 = { <?, ?, ?, ?, ?, ?> }

Example 1 (Positive): <Sunny, Warm, Normal, Strong, Warm, Same>

- G0 is consistent with it, so there is no change: **G1 = G0**.
- Generalize S minimally to cover the example: **S1 = { <Sunny, Warm, Normal, Strong, Warm, Same> }**

Example 2 (Positive): <Sunny, Warm, High, Strong, Warm, Same>

- G is consistent, so **G2 = G1**.
- Humidity differs (Normal vs High), so replace it with ?: **S2 = { <Sunny, Warm, ?, Strong, Warm, Same> }**

Example 3 (Negative): <Rainy, Cold, High, Strong, Warm, Change>

- S2 does not cover this negative example, so **S3 = S2**.
- Specialize G minimally so it excludes the example, keeping only specializations that are **more general than S2**:

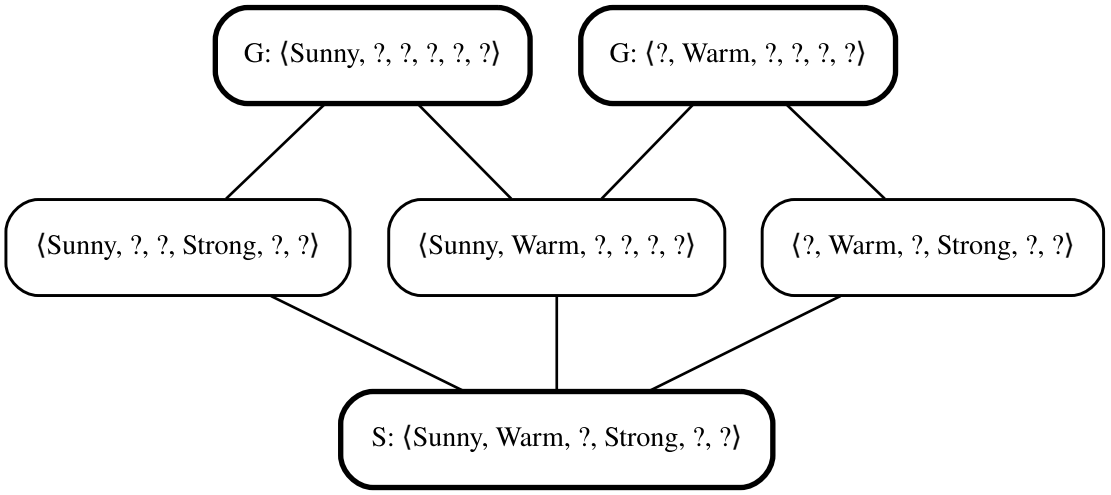
Attribute	Value in negative example	Specialization	More general than S2?	Keep?
Sky	Rainy	<Sunny, ?, ?, ?, ?, ?>	Yes	**Yes**
AirTemp	Cold	<?, Warm, ?, ?, ?, ?>	Yes	**Yes**
Humidity	High	<?, ?, Normal, ?, ?, ?>	No (S2 has ?)	No
Wind	Strong	<?, ?, ?, Light, ?, ?>	No (S2 has Strong)	No
Water	Warm	<?, ?, ?, ?, Cool, ?>	No (S2 has Warm)	No
Forecast	Change	<?, ?, ?, ?, ?, Same>	Yes	**Yes**

- **G3 = { <Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>, <?, ?, ?, ?, ?, Same> }**

Example 4 (Positive): <Sunny, Warm, High, Strong, Cool, Change>

- Remove from G any hypothesis that does **not** cover this positive example. <?, ?, ?, ?, ?, Same> fails (Forecast = Change), so it is **removed**. **G4 = { <Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?> }**
- Generalize S: Water (Warm vs Cool) and Forecast (Same vs Change) differ, so both become ?. **S4 = { <Sunny, Warm, ?, Strong, ?, ?> }**

Final Version Space (all 6 consistent hypotheses)



Final version space: the S boundary, the three hypotheses between, and the G boundary

Answer:

- **Maximally specific hypothesis: S = <Sunny, Warm, ?, Strong, ?, ?>**
- **Maximally general hypotheses: G = { <Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?> }**

4. Key Characteristics - Find-S vs. Candidate Elimination

Basis	Find-S	Candidate Elimination
Output	**One** maximally specific hypothesis	The whole **version space** (S and G boundaries)
Uses negative examples	No (ignores them)	**Yes** (to specialize G)
Detects inconsistent data	No	Yes (S and G become empty or cross)
Can tell when learning is complete	No	**Yes** (when S = G)
Memory	Low	Higher (stores two sets)

5. Advantages & Disadvantages

Advantages

- Represents **every consistent hypothesis** compactly with just two boundaries.
- Uses **both positive and negative** examples.
- Can **suggest the most informative next query** (an example that splits the version space).

Disadvantages

- **Very sensitive to noise**: one mislabelled example can make the version space **empty**.
- Assumes the target concept is **inside the hypothesis space** (conjunctions of attributes only).
- The size of G can **grow exponentially** in the worst case.

6. Conclusion / Application

Candidate Elimination illustrates **concept learning and version spaces**, and it is the theoretical basis of **active learning**, where the learner chooses which examples to label next.

Unit 2: Regression, Bayesian Learning & SVM

Q1. Short Definitions (2 Marks Each)

(a) Define the Maximum Likelihood Hypothesis.

- The **Maximum Likelihood (ML) hypothesis** is the hypothesis h in hypothesis space H that **maximizes the probability of the observed training data D**:

$$h_{ML} = \arg \max_{h \in H} P(D \mid h)$$

- It is a special case of the **MAP (Maximum A Posteriori)** hypothesis, `h_MAP = argmax P(D|h)·P(h)`, in which **all hypotheses are assumed equally likely a priori**, so P(h) is the same for all h.
- *Result:* under Gaussian noise, the ML hypothesis is the one that **minimizes the sum of squared errors**. This is the basis of least-squares regression.

(b) Two real-world applications of Logistic Regression.

1. **Medical diagnosis:** predicting whether a patient **has a disease (Yes/No)**, such as diabetes or heart disease, from features like age, BMI, blood pressure and glucose level.

2. **Email spam detection:** classifying an email as **spam or not spam** from word frequencies and sender features.

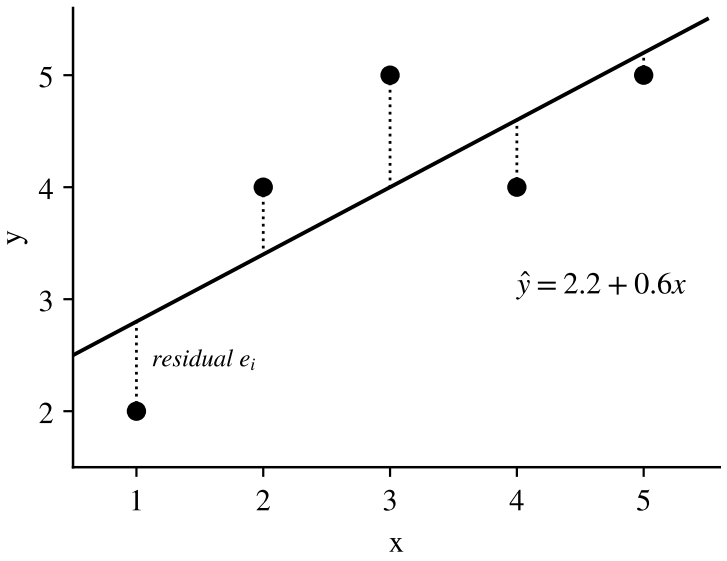
- *Others:* credit-card **fraud detection**, **customer churn** prediction, loan default prediction.

Q2. Derive the least squares solution for Linear Regression. (7 Marks)

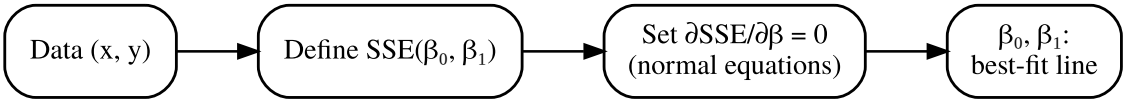
1. Introduction / Definition

Linear Regression models the relationship between an input x and a continuous output y as a **straight line**, `ŷ = β0 + β1·x`. The **Method of Least Squares** finds the values of β0 and β1 that **minimize the Sum of Squared Errors (SSE)** between the actual values yi and the predicted values ŷi.

2. Block Diagram



Least-squares line through the five data points; each residual is the vertical gap between a point and the line



3. Core Explanation / Working - Derivation

Step 1 - Define the cost function (SSE)

$$SSE\left(\beta_0, \beta_1\right) = \sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2 = \sum_{i=1}^{n} \left(y_i - \beta_0 - \beta_1 x_i\right)^2$$

Step 2 - Partial derivative with respect to β0, set to zero

$$\frac{\partial SSE}{\partial \beta_0} = -2\sum \left(y_i - \beta_0 - \beta_1 x_i\right) = 0 \quad \Rightarrow \quad \sum y_i = n\beta_0 + \beta_1 \sum x_i \qquad \text{(Normal Eq. 1)}$$

Dividing by n:

$$\bar{y} = \beta_0 + \beta_1 \bar{x} \quad \Rightarrow \quad \beta_0 = \bar{y} - \beta_1 \bar{x} \qquad \text{(A)}$$

Step 3 - Partial derivative with respect to β1, set to zero

$$\frac{\partial SSE}{\partial \beta_1} = -2 \sum x_i \left(y_i - \beta_0 - \beta_1 x_i \right) = 0 \quad \Rightarrow \quad \sum x_i y_i = \beta_0 \sum x_i + \beta_1 \sum x_i^2 \qquad \text{(Normal Eq. 2)}$$

Step 4 - Substitute (A) into Normal Eq. 2

$$\sum x_i y_i = \left(\bar{y} - \beta_1 \bar{x} \right) \sum x_i + \beta_1 \sum x_i^2$$

$$\sum x_i y_i - \bar{y} \sum x_i = \beta_1 \left(\sum x_i^2 - \bar{x} \sum x_i \right)$$

Using $\sum x_i = n\bar{x}$:

$$\sum x_i y_i - n\bar{x}\bar{y} = \beta_1 \left(\sum x_i^2 - n\bar{x}^2 \right)$$

$$\beta_1 = \frac{\sum x_i y_i - n\bar{x}\bar{y}}{\sum x_i^2 - n\bar{x}^2} = \frac{\sum \left(x_i - \bar{x} \right) \left(y_i - \bar{y} \right)}{\sum \left(x_i - \bar{x} \right)^2} = \frac{\mathrm{Cov}(x, y)}{\mathrm{Var}(x)}$$

Final least squares solution:

$$\beta_1 = \frac{\sum \left(x_i - \bar{x} \right) \left(y_i - \bar{y} \right)}{\sum \left(x_i - \bar{x} \right)^2}, \qquad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

Step 5 - Matrix form (multiple linear regression)

$$\text{Model: } \mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \qquad \text{Cost: } J(\boldsymbol{\beta}) = (\mathbf{y} - X\boldsymbol{\beta})^T (\mathbf{y} - X\boldsymbol{\beta})$$

$$\frac{\partial J}{\partial \boldsymbol{\beta}} = -2X^T\mathbf{y} + 2X^TX\boldsymbol{\beta} = 0 \quad \Rightarrow \quad X^TX\boldsymbol{\beta} = X^T\mathbf{y} \quad \Rightarrow \quad \boldsymbol{\beta} = \left(X^TX \right)^{-1} X^T\mathbf{y}$$

Numerical example: x = [1, 2, 3, 4, 5], y = [2, 4, 5, 4, 5]

x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
1	2	−2	−2	4	4
2	4	−1	0	0	1
3	5	0	1	0	0
4	4	1	0	0	1
5	5	2	1	2	4
Σ				**6**	**10**

- $\bar{x} = 3$, $\bar{y} = 4$
- **$\beta 1 = 6 / 10 = 0.6$, $\beta 0 = 4 - 0.6 \times 3 = 2.2$**
- **Regression line: $\hat{y} = 2.2 + 0.6x$**. For example, at x = 6, $\hat{y} = 5.8$.

4. Key Characteristics - Normal Equation vs. Gradient Descent

Basis	Least Squares (Normal Equation)	Gradient Descent
Method	**Closed-form** (direct formula)	**Iterative**
Learning rate	Not needed	Must choose α
Complexity	$O(n^3)$ for the $(X^TX)^{-1}$ inversion	$O(k \cdot m \cdot n)$ for k iterations
Large number of features	Slow	**Scales well**
Requirement	X^TX must be invertible	Cost should be convex for a guaranteed optimum

5. Advantages & Disadvantages

Advantages

- Gives an **exact, unique optimal solution** in one step.
- **Simple to compute** and interpret ($\beta 1$ is the slope, $\beta 0$ the intercept).
- Statistically optimal (it is the ML estimate) under **Gaussian noise**.

Disadvantages

- **Sensitive to outliers**, because squaring magnifies large errors.
- Assumes a **linear relationship** and **independent, constant-variance errors**.
- Matrix inversion is **costly or unstable** with many or correlated features (multicollinearity).

6. Conclusion / Application

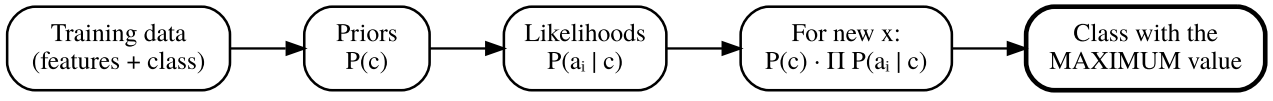
Least squares regression is used for **sales forecasting, house-price prediction, trend analysis in economics, and calibrating scientific instruments**.

Q3. Explain the Naive Bayes Classifier with a mathematical example. (7 Marks)

1. Introduction / Definition

The **Naive Bayes Classifier** is a **probabilistic supervised classifier** based on **Bayes' Theorem**. It makes the **"naive" assumption that all features are conditionally independent of each other, given the class**. It predicts the class with the **highest posterior probability**.

2. Block Diagram



Naive Bayes training and prediction

Bayes' theorem: $$\underbrace{P(c \mid x)}_{\text{posterior}} = \frac{\overbrace{P(x \mid c)}^{\text{likelihood}}\ \overbrace{P(c)}^{\text{prior}}}{\underbrace{P(x)}_{\text{evidence}}}$$

3. Core Explanation / Working

Bayes' Theorem

$$P(c \mid x) = \frac{P(x \mid c)\, P(c)}{P(x)}$$

Naive independence assumption: for x = <a1, a2, …, an>,

$$P(a_1, a_2, \ldots, a_n \mid c) = P(a_1 \mid c)\, P(a_2 \mid c) \cdots P(a_n \mid c)$$

$$c_{NB} = \arg\max_{c \in C} P(c) \prod_i P(a_i \mid c) \qquad (P(x) \text{ is the same for all classes, so it is dropped})$$

Mathematical example - the PlayTennis dataset (14 days: 9 Yes, 5 No)

Classify the new day x = <Outlook = Sunny, Temp = Cool, Humidity = High, Wind = Strong>.

Step 1 - Prior probabilities

- P(Yes) = 9/14 = **0.643**
- P(No) = 5/14 = **0.357**

Step 2 - Likelihoods from the frequency tables

Feature = value	Count with Yes	P(value | Yes)	Count with No	P(value | No)
Outlook = Sunny	2	2/9 = 0.222	3	3/5 = 0.600
Temp = Cool	3	3/9 = 0.333	1	1/5 = 0.200
Humidity = High	3	3/9 = 0.333	4	4/5 = 0.800
Wind = Strong	3	3/9 = 0.333	3	3/5 = 0.600

Step 3 - Compute P(c) × Π P(ai | c)

$$\text{YES: } P(\text{Yes})\, P(\text{Sunny}|\text{Yes})\, P(\text{Cool}|\text{Yes})\, P(\text{High}|\text{Yes})\, P(\text{Strong}|\text{Yes}) = \frac{9}{14} \cdot \frac{2}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} \cdot \frac{3}{9} = 0.0053$$

$$\text{NO: } P(\text{No})\, P(\text{Sunny}|\text{No})\, P(\text{Cool}|\text{No})\, P(\text{High}|\text{No})\, P(\text{Strong}|\text{No}) = \frac{5}{14} \cdot \frac{3}{5} \cdot \frac{1}{5} \cdot \frac{4}{5} \cdot \frac{3}{5} = 0.0206$$

Step 4 - Normalize (optional) and decide

$$P(\text{No} \mid x) = \frac{0.0206}{0.0206 + 0.0053} = 0.795\ (79.5\%), \qquad P(\text{Yes} \mid x) = \frac{0.0053}{0.0206 + 0.0053} = 0.205\ (20.5\%)$$

Prediction: PlayTennis = NO.

Zero-frequency problem and Laplace smoothing

- If a feature value never occurs with a class, P = 0 and the whole product becomes 0.
- **Laplace (add-one) smoothing:** `P(ai | c) = (count(ai, c) + 1) / (count(c) + k)`, where k is the number of distinct values of that feature.

4. Key Characteristics - Types of Naive Bayes

Type	Feature type	Likelihood model	Typical use
Gaussian NB	Continuous	Normal distribution (mean, variance)	Iris and medical data
Multinomial NB	Counts (word frequencies)	Multinomial	**Text classification**
Bernoulli NB	Binary (0/1)	Bernoulli	Spam (word present or absent)

5. Advantages & Disadvantages

Advantages

- **Very fast** to train and predict, in O(n·d) time.
- Works well with **small datasets** and **high-dimensional text** data.
- Handles **multi-class** problems naturally and gives probability estimates.

Disadvantages

- The **independence assumption is rarely true** (for example Humidity and Temperature are correlated).
- The **zero-frequency problem** needs smoothing.
- Its probability estimates are often **poorly calibrated**, even when the class prediction is correct.

6. Conclusion / Application

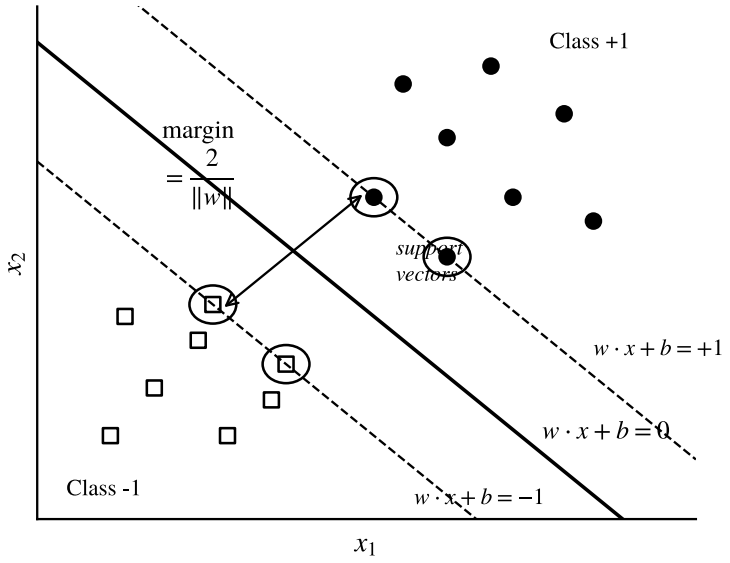
Naive Bayes is widely used for **spam filtering, sentiment analysis, news categorization and real-time medical diagnosis**.

Q4. Discuss the architecture, margin and core issues of a Support Vector Machine (SVM). (7 Marks)

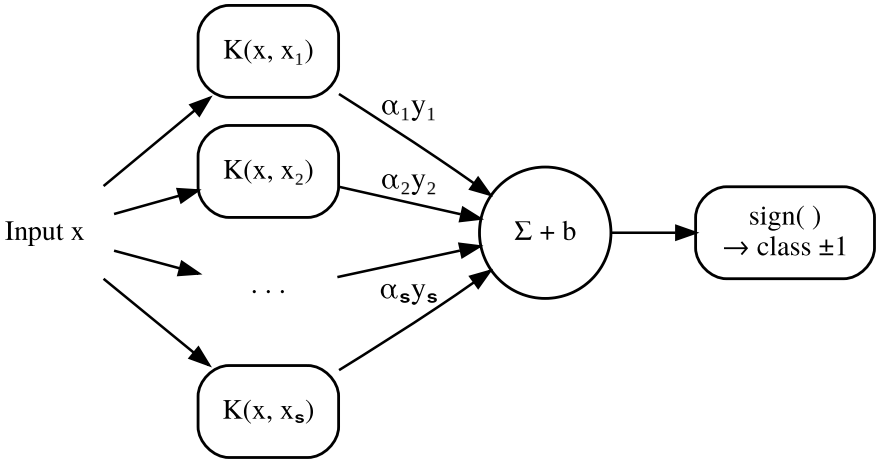
1. Introduction / Definition

A **Support Vector Machine (SVM)** is a **supervised learning algorithm** for classification and regression. It finds the **optimal separating hyperplane** that divides the classes with the **maximum margin**. The data points closest to the hyperplane, called **support vectors**, alone decide its position.

2. Block Diagram - Maximum Margin Hyperplane



Maximum-margin hyperplane with support vectors (circled)



SVM viewed as a network: one kernel unit per support vector

3. Core Explanation / Working

(a) The Hyperplane

- In n dimensions, a hyperplane is `w·x + b = 0`, where **w** is the weight (normal) vector and **b** is the bias.
- Classification rule: **f(x) = sign(w·x + b)**, so +1 is one class and −1 is the other.

(b) The Margin

- The **margin** is the distance between the two parallel boundary hyperplanes `w·x + b = +1` and `w·x + b = −1`:

$$\text{Margin} = \frac{2}{\| \mathbf{w} \|}$$

- **Maximizing the margin** is the same as **minimizing ||w||**. A larger margin gives **better generalization**.

(c) Optimization Problem (Hard Margin)

$$\min_{\mathbf{w},\, b} \quad \frac{1}{2} \| \mathbf{w} \|^2 \qquad \text{subject to} \qquad y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b \right) \geq 1 \quad \forall i$$

- This is solved with **Lagrange multipliers αi**. In the dual form, only the points with **αi > 0** affect the solution. These points are the **support vectors**.

(d) Soft Margin (for non-separable or noisy data)

$$\min \quad \frac{1}{2} \| \mathbf{w} \|^2 \; + C \sum_i \xi_i \qquad \text{subject to} \qquad y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b \right) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- The **slack variables ξi** allow some points to be misclassified or to fall inside the margin.
- **C** controls the trade-off. A **large C** gives a narrow margin and few errors (risk of overfitting). A **small C** gives a wide margin and more errors (better generalization).

(e) Kernel Trick (non-linear SVM)

- The data is mapped into a **higher-dimensional space** φ(x), where it becomes **linearly separable**, without computing φ explicitly. Only the kernel `K(xi, xj) = φ(xi)·φ(xj)` is needed.
- **Common kernels:**
  - Linear: `K = xi·xj`
  - Polynomial: `K = (xi·xj + c)^d`
  - **RBF / Gaussian:** `K = exp(−γ ||xi − xj||²)`
  - Sigmoid: `K = tanh(κ xi·xj + c)`

(f) Core Issues of SVM

1. **Choice of kernel:** there is no universal rule. A wrong kernel gives poor accuracy.
2. **Hyper-parameter tuning:** C and γ must be tuned (by grid search with cross-validation).
3. **Computational cost:** training takes **$O(n^2)$ to $O(n^3)$**, so it is slow for very large datasets.
4. **Feature scaling:** SVM is sensitive to scale, so features must be **normalized**.
5. **Multi-class classification:** SVM is binary by nature. It needs **One-vs-One** or **One-vs-All** schemes.
6. **No direct probability output:** this needs extra calibration (Platt scaling).
7. **Interpretability:** with non-linear kernels, the model is hard to interpret.

4. Key Characteristics - Hard Margin vs. Soft Margin SVM

Basis	Hard Margin SVM	Soft Margin SVM
Data assumption	Perfectly linearly separable	Overlapping or noisy data allowed
Misclassification	**Not allowed**	Allowed (slack ξ)
Parameter	None	Penalty **C**
Sensitivity to outliers	**Very high**	Lower
Practical use	Rare	**Standard**

5. Advantages & Disadvantages

Advantages

- **Effective in high-dimensional spaces**, even when there are more features than samples.
- **Memory efficient**, because only the support vectors are stored.

- **Kernel trick** handles complex non-linear boundaries. Maximum margin gives **strong generalization**.

Disadvantages

- **Slow on large datasets** and sensitive to the choice of kernel and parameters.
- **No probabilistic output**, and the model is hard to interpret.
- Performs poorly on **heavily overlapping and noisy** classes.

6. Conclusion / Application

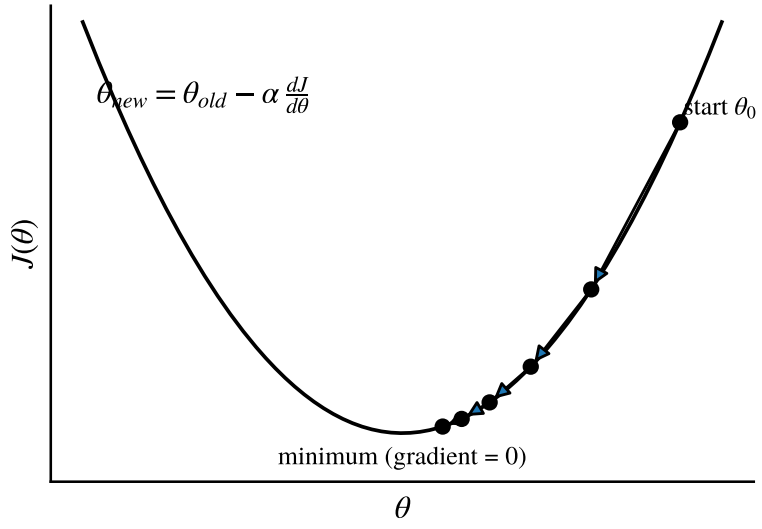
SVMs are used in **handwriting recognition, face detection, text and email categorization, bioinformatics (protein classification) and image classification**.

Q5. Explain Gradient Descent and how it minimizes the cost function. (7 Marks)

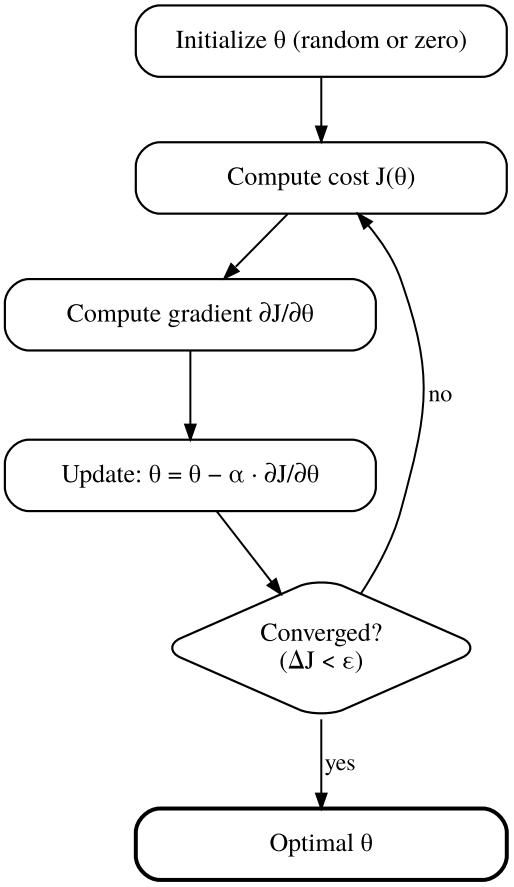
1. Introduction / Definition

Gradient Descent is an **iterative first-order optimization algorithm** that finds the **minimum of a cost function J(θ)**. It repeatedly moves the parameters **in the direction of the negative gradient**, which is the direction of steepest decrease, until it converges to a (local or global) minimum.

2. Block Diagram - Cost Curve and Algorithm Flow



Gradient descent steps down the cost curve towards the minimum; steps get smaller as the slope flattens



Gradient descent loop

3. Core Explanation / Working

(a) Cost function (Linear Regression, Mean Squared Error)

$$h_\theta(x) = \theta_0 + \theta_1 x, \qquad J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x_i) - y_i)^2$$

(b) Update rule (applied to all θj at the same time)

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

For linear regression:

$$\theta_0 := \theta_0 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x_i) - y_i), \qquad \theta_1 := \theta_1 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x_i) - y_i)\,x_i$$

(c) Why it minimizes the cost

- The **gradient ∂J/∂θ points in the direction of steepest increase** of J. Subtracting it moves θ **downhill**.
- **Near the minimum the gradient becomes small**, so the steps automatically get smaller.
- At the minimum, **∂J/∂θ = 0**, so θ stops changing (convergence).
- The MSE cost of linear regression is **convex** (bowl-shaped), so gradient descent reaches the **global minimum**.

(d) Role of the learning rate α

- **α too small:** very slow convergence (many iterations).
- **α too large:** it overshoots the minimum and may **diverge**.
- **α just right:** fast and stable convergence.

(e) Numerical example: minimize $J(\theta) = \theta^2$, with $dJ/d\theta = 2\theta$, $\theta 0 = 4$ and $\alpha = 0.1$.

Iteration	θ (old)	Gradient 2θ	θ_new = θ − 0.1(2θ)	J(θ_new)
1	4.000	8.000	3.200	10.24
2	3.200	6.400	2.560	6.55
3	2.560	5.120	2.048	4.19
4	2.048	4.096	1.638	2.68
…	…	…	→ 0	→ 0 (minimum)

4. Key Characteristics - Types of Gradient Descent

Type	Data used per update	Speed	Convergence	Memory
Batch GD	Entire dataset	Slow per step	Smooth, stable	High
Stochastic GD (SGD)	**One** random sample	Fast	Noisy (can escape local minima)	Low
Mini-batch GD	Small batch (for example 32 or 64)	**Fast**	Fairly stable	Moderate

Improved variants: Momentum, RMSProp, **Adam** (adaptive learning rates).

5. Advantages & Disadvantages

Advantages

- **Simple and general**: works for any differentiable cost function.
- **Scales to huge datasets** and millions of parameters (SGD and mini-batch).
- It is the **core training algorithm** of neural networks and deep learning.

Disadvantages

- It can get **stuck in local minima or saddle points** on non-convex functions.
- It is **sensitive to the learning rate** and to feature scaling.
- It may need **many iterations** to converge.

6. Conclusion / Application

Gradient descent is used to train **linear and logistic regression, neural networks (with backpropagation), and deep learning models like CNNs and GPT**.

Unit 3: Decision Trees & Instance-Based Learning

Q1. Short Definitions (2 Marks Each)

(a) Define Entropy in the context of Decision Trees.

- **Entropy** measures the **impurity (uncertainty or randomness)** of a set of examples S with respect to the class labels:

$$\text{Entropy}(S) = -\sum_{i=1}^{c} p_i \log_2 p_i \qquad \text{(two classes: Entropy}(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-\text{)}$$

- **Entropy = 0** when all examples belong to **one class** (pure). **Entropy = 1** (for 2 classes) when the classes are **equally mixed** (50:50).
- *Example:* S = [9+, 5−] gives Entropy = −(9/14)log2(9/14) − (5/14)log2(5/14) = **0.940**.
- ID3 uses entropy to compute **Information Gain** and choose the best splitting attribute.

(b) What is the core concept of Instance-Based Learning?

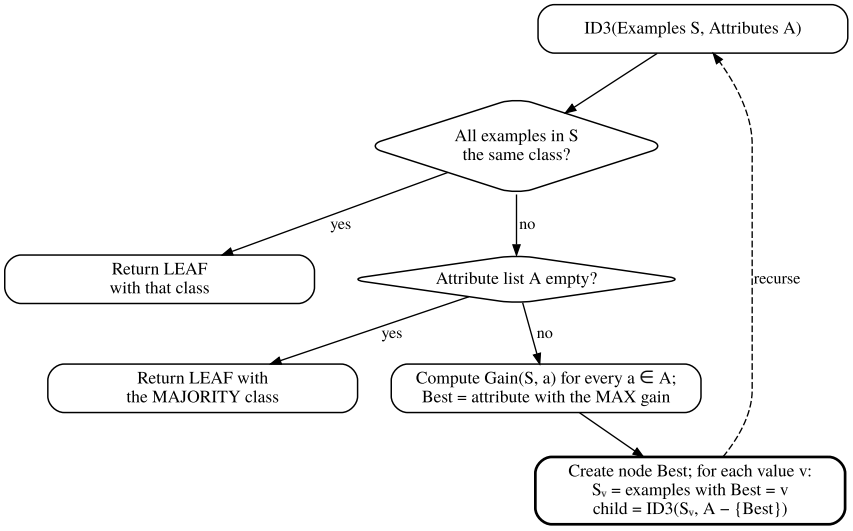
- **Instance-Based Learning (IBL)**, also called **lazy learning** or memory-based learning, **does not build an explicit model** during training. It simply **stores all the training examples**.
- When a **new query** arrives, it finds the **most similar stored instances** (using a distance measure such as Euclidean distance) and predicts the output from them. The **generalization is postponed until query time**.
- *Examples:* **k-Nearest Neighbour (k-NN)**, locally weighted regression, Case-Based Reasoning (CBR), RBF networks.

Q2. Trace the ID3 algorithm on a given dataset using Information Gain. (7 Marks)

1. Introduction / Definition

ID3 (Iterative Dichotomiser 3), developed by **J. Ross Quinlan**, builds a **decision tree top-down and greedily**. At each node it selects the attribute with the **highest Information Gain**, which is the largest reduction in entropy. It splits the data on that attribute and repeats the process until the subsets are **pure** or no attributes are left.

2. Flowchart - ID3 Algorithm



ID3 algorithm

$$\text{Gain}\,(S, A) \;=\; \text{Entropy}\,(S) \;-\; \sum_{v \,\in\, \text{Values}\,(A)} \frac{|S_v|}{|S|}\,\text{Entropy}\,(S_v)$$

3. Core Explanation / Working

Dataset: PlayTennis (14 examples)

Day	Outlook	Temp	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Step 1 - Entropy of the whole set: S = [9+, 5−]

$$\text{Entropy}\,(S) \;=\; -\,\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.410 + 0.530 = 0.940$$

Step 2 - Information Gain of each attribute at the root

Attribute	Value: [+, −] → Entropy	Weighted entropy	Gain
Outlook	Sunny [2,3] → 0.971; Overcast [4,0] → 0; Rain [3,2] → 0.971	(5/14)(0.971) + 0 + (5/14)(0.971) = 0.694	**0.940 − 0.694 = 0.246**
Humidity	High [3,4] → 0.985; Normal [6,1] → 0.592	(7/14)(0.985) + (7/14)(0.592) = 0.789	0.151
Wind	Weak [6,2] → 0.811; Strong [3,3] → 1.000	(8/14)(0.811) + (6/14)(1.0) = 0.892	0.048
Temp	Hot [2,2] → 1.0; Mild [4,2] → 0.918; Cool [3,1] → 0.811	(4/14)(1) + (6/14)(0.918) + (4/14)(0.811) = 0.911	0.029

- **Outlook has the highest gain (0.246), so it becomes the ROOT node.**
- **Overcast** is pure (all Yes), so it becomes a **leaf = Yes**.

Step 3 - Sunny branch: S_sunny = {D1, D2, D8, D9, D11} = [2+, 3−], Entropy = 0.971

Attribute	Split	Gain
Humidity	High {D1, D2, D8} = [0,3] → 0; Normal {D9, D11} = [2,0] → 0	**0.971 − 0 = 0.971**
Temp	Hot [0,2] → 0; Mild [1,1] → 1; Cool [1,0] → 0	0.971 − (2/5)(1) = 0.571
Wind	Weak [1,2] → 0.918; Strong [1,1] → 1	0.971 − 0.551 − 0.400 = 0.020

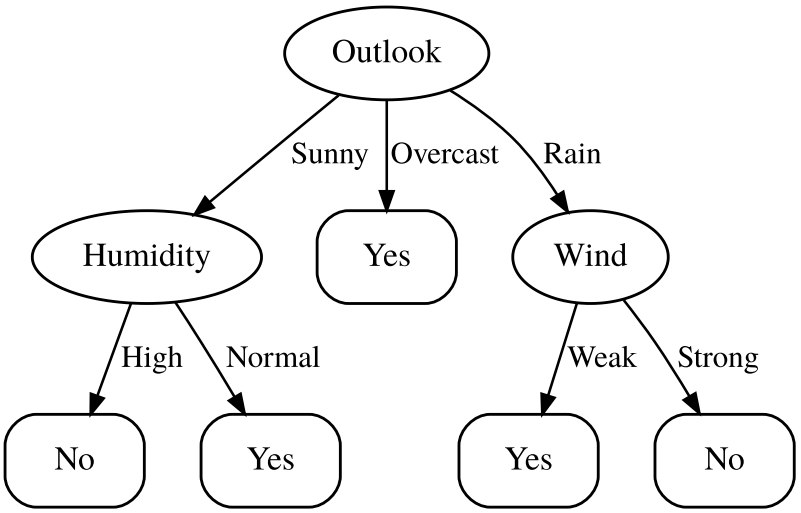
- **Humidity is chosen for the Sunny branch.** High gives No and Normal gives Yes (both pure).

Step 4 - Rain branch: S_rain = {D4, D5, D6, D10, D14} = [3+, 2−], Entropy = 0.971

Attribute	Split	Gain
Wind	Weak {D4, D5, D10} = [3,0] → 0; Strong {D6, D14} = [0,2] → 0	**0.971**
Humidity	High [1,1] → 1; Normal [2,1] → 0.918	0.020
Temp	Mild [2,1] → 0.918; Cool [1,1] → 1	0.020

- **Wind is chosen for the Rain branch.** Weak gives Yes and Strong gives No.

Final Decision Tree



Final decision tree for PlayTennis

Rules extracted from the tree

- IF Outlook = Sunny AND Humidity = High THEN Play = No
- IF Outlook = Sunny AND Humidity = Normal THEN Play = Yes
- IF Outlook = Overcast THEN Play = Yes
- IF Outlook = Rain AND Wind = Weak THEN Play = Yes
- IF Outlook = Rain AND Wind = Strong THEN Play = No
- *Note:* Temperature is **never used**. It was not informative.

4. Key Characteristics - ID3 vs. C4.5 vs. CART

Basis	ID3	C4.5	CART
Split criterion	Information Gain	**Gain Ratio**	**Gini Index**
Attribute types	Categorical only	Categorical + continuous	Categorical + continuous
Missing values	Not handled	Handled	Handled
Pruning	No	Yes (error-based)	Yes (cost-complexity)
Tree type	Multi-way	Multi-way	**Binary**

5. Advantages & Disadvantages

Advantages

- Builds an **easy-to-interpret** tree and readable **if-then rules**.
- **Fast**, greedy construction. Short trees follow **Occam's Razor**.
- **No feature scaling** is needed, and irrelevant attributes are ignored automatically (like Temp here).

Disadvantages

- **Overfits** noisy data (there is no pruning in basic ID3).
- **Biased toward attributes with many values** (for example a "Day" attribute would get the highest gain).
- Handles **only categorical** attributes, and greedy search may miss the globally best tree.

6. Conclusion / Application

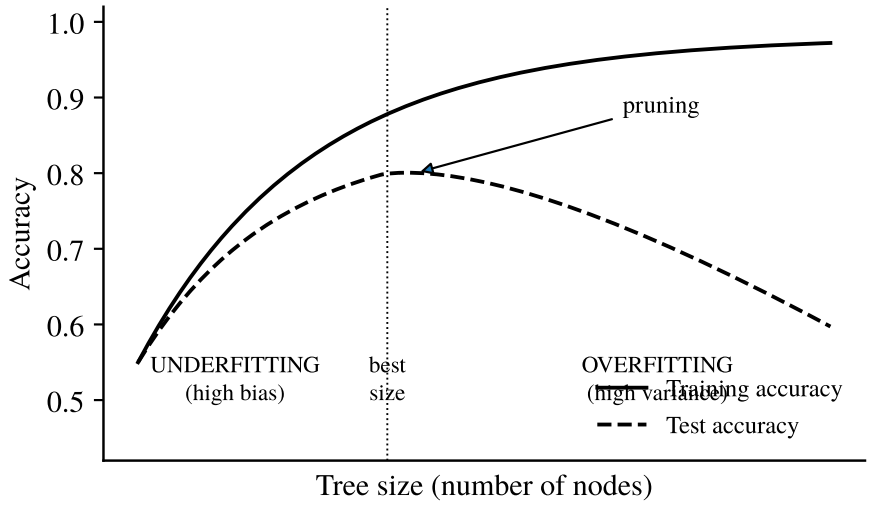
ID3-style decision trees are used for **loan approval, medical diagnosis, customer-churn prediction and fault diagnosis**, where explainable decisions are needed.

Q3. Discuss overfitting and underfitting in decision trees and explain tree pruning. (7 Marks)

1. Introduction / Definition

Overfitting happens when a decision tree **grows too deep** and **memorizes the training data, including its noise**. It then has high training accuracy but **poor accuracy on unseen data**. **Underfitting** happens when the tree is **too shallow** to capture the real pattern, so accuracy is poor on **both** training and test data. **Pruning removes branches that do not improve generalization**, which reduces overfitting.

2. Block Diagram - Accuracy vs. Tree Size



Accuracy vs. tree size: training accuracy keeps rising while test accuracy peaks and then falls (overfitting); pruning moves the tree back to the best size

3. Core Explanation / Working

(a) Overfitting

- **Definition (Mitchell):** a hypothesis h overfits if there is another hypothesis h' with **higher training error** but **lower error over the entire distribution** of instances.
- **Causes:**
  - **Noise** in the training data (mislabelled examples).
  - **Too few training examples** for the tree's complexity.
  - Growing the tree until **every leaf is pure**.
  - Many **irrelevant attributes** that split by chance.
- **Symptoms:** very deep tree, many leaves with only 1 or 2 examples, training accuracy near 100% but low test accuracy.

(b) Underfitting

- The tree is **too simple** (for example only one split, a "decision stump").
- **Causes:** too much pruning, a very small max-depth, or a very high minimum-samples-per-leaf.
- **Symptoms: high bias** and low accuracy on both training and test data.

(c) Tree Pruning - the Remedy

(i) Pre-pruning (Early Stopping)

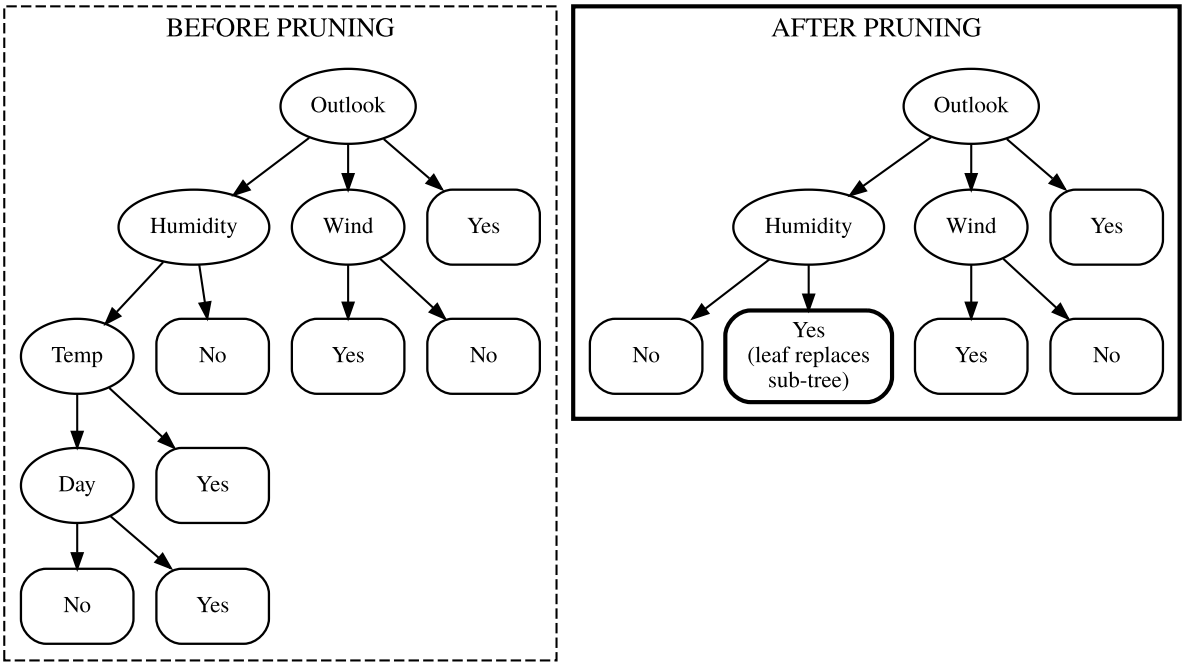
- **Stop growing** the tree before it becomes perfectly pure, using rules such as:
  - **Maximum depth** reached.
  - **Minimum number of samples** in a node (for example fewer than 5).
  - **Information gain below a threshold**.
  - A **chi-square test** shows the split is not statistically significant.
- *Problem:* the "horizon effect". A split that looks weak now might lead to useful splits later.

(ii) Post-pruning (Grow, then Prune)

- Grow the **full tree** first, then remove sub-trees that do not help. This is **more reliable** in practice.

(iii) Post-pruning Methods

1. **Reduced Error Pruning (REP):**
   - Split the data into a **training set** and a **validation set**.
   - For each internal node, **replace its sub-tree with a leaf** (the majority class).
   - Keep the change if the **validation accuracy does not decrease**.
   - Repeat until further pruning hurts accuracy.
2. **Rule Post-Pruning (used in C4.5):**
   - Convert the tree into **if-then rules** (one per root-to-leaf path).
   - **Remove preconditions** from each rule if this improves the estimated accuracy.
   - **Sort the rules** by accuracy and apply them in that order.
3. **Cost-Complexity Pruning (used in CART):**
   - Minimize $\mathbf{R_\alpha(T) = R(T) + \alpha \cdot |T|}$, where R(T) is the error, |T| is the number of leaves and α is the complexity penalty.
   - Choose α by **cross-validation**.



Before and after pruning: the noisy sub-tree under Humidity = Normal is replaced by its majority leaf

4. Key Characteristics - Overfitting vs. Underfitting

Basis	Underfitting	Overfitting
Model complexity	Too simple (shallow tree)	Too complex (deep tree)
Training accuracy	**Low**	**Very high** (around 100%)
Test accuracy	Low	**Low**
Bias / Variance	**High bias**, low variance	Low bias, **high variance**
Cause	Over-pruning, few features	Noise, small data, no pruning
Remedy	Grow a deeper tree, add features	**Pruning**, more data, ensembles (Random Forest)

5. Advantages & Disadvantages (of pruning)

Advantages

- **Improves generalization**, which means better accuracy on unseen data.
- Gives a **smaller, simpler tree** that is easier to interpret (Occam's Razor).
- **Reduces the effect of noise** and outliers.

Disadvantages

- Post-pruning **needs a separate validation set**, which leaves less training data.
- Pruning too much leads to **underfitting**, and choosing the thresholds or α needs tuning.

6. Conclusion / Application

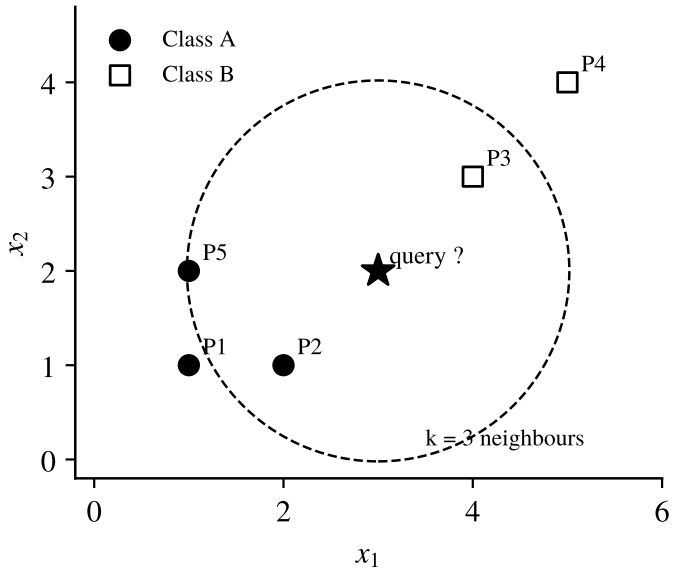
Pruned decision trees (C4.5, CART) and their ensembles (**Random Forest, XGBoost**) are used in **credit scoring, medical diagnosis and fraud detection**, where generalization matters.

Q4. Explain the k-Nearest Neighbour (k-NN) algorithm and its limitations. (7 Marks)

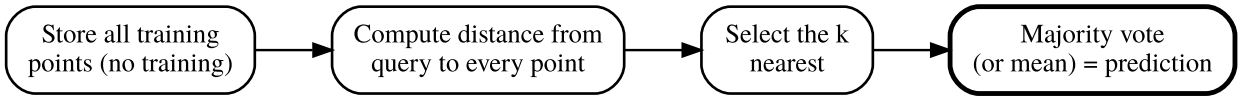
1. Introduction / Definition

k-Nearest Neighbour (k-NN) is a **supervised, non-parametric, instance-based (lazy)** learning algorithm. To classify a new point, it finds the **k closest training examples** (using a distance measure) and assigns the **majority class** among them. For regression, it predicts the **average** of their values.

2. Block Diagram - k-NN Classification



k-NN with k = 3: the three nearest neighbours of the query (?) are A, A and B, so the majority class is A



3. Core Explanation / Working

Algorithm

1. **Store** all training examples (x_i, y_i). There is no model-building step.
2. Choose the number of neighbours **k** (usually odd, for example 3 or 5, to avoid ties).
3. For a query point x_q, **compute its distance** to every training point.
4. **Sort** the distances and select the **k nearest** neighbours.
5. **Classification:** return the **majority class** among the k neighbours. **Regression:** return the **mean** of their y values.
6. *(Distance-weighted k-NN:* closer neighbours get a larger weight, w_i = 1/d(x_q, x_i)².)

Distance measures

$$\text{Euclidean: } d(x,y) = \sqrt{\sum_i (x_i - y_i)^2} \qquad \text{Manhattan: } d(x,y) = \sum_i |x_i - y_i|$$

$$\text{Minkowski: } d(x,y) = \left(\sum_i |x_i - y_i|^p\right)^{1/p} \qquad (p = 1 \text{ Manhattan},\ p = 2 \text{ Euclidean})$$

Numerical example: classify the query **Q = (3, 2)** with **k = 3**.

Point	(x1, x2)	Class	Euclidean distance to (3, 2)	Rank
P1	(1, 1)	A	√(4 + 1) = 2.236	4
P2	(2, 1)	A	√(1 + 1) = **1.414**	**1**
P3	(4, 3)	B	√(1 + 1) = **1.414**	**2**
P4	(5, 4)	B	√(4 + 4) = 2.828	5
P5	(1, 2)	A	√(4 + 0) = **2.000**	**3**

- The 3 nearest neighbours are P2 (A), P3 (B) and P5 (A). **Votes: A = 2, B = 1.**
- **Prediction: Class A.**

Choosing k

- **Small k (k = 1):** sensitive to noise, so it overfits.
- **Large k:** a smoother boundary, but it may underfit and ignore local structure.
- Choose k by **cross-validation**. A common rule of thumb is k ≈ √n.

Limitations of k-NN

1. **High prediction cost:** distances to **all n training points** must be computed for each query, which takes O(n·d) time. It is slow for large datasets.
2. **High memory requirement:** the **entire training set** must be stored.
3. **Curse of dimensionality:** in high dimensions, all points become almost equally distant, so “nearest” loses meaning.
4. **Sensitive to feature scaling:** features with large ranges (for example salary) dominate the distance, so **normalization** is required.
5. **Sensitive to irrelevant features and noise:** all features count equally in the distance.
6. **Choice of k and of the distance metric** strongly affects the results.
7. **Imbalanced data:** the majority class dominates the votes.

4. Key Characteristics - k-NN (Lazy) vs. Decision Tree (Eager)

Basis	k-NN (Lazy Learner)	Decision Tree (Eager Learner)
Training phase	**None** (just stores the data)	Builds the model in advance
Prediction speed	**Slow** (distance to all points)	**Fast** (follow one path)
Memory	Stores **all** training data	Stores only the tree
Decision boundary	Complex and local (adapts to the data)	Axis-parallel rectangles
Feature scaling needed	**Yes**	No
Interpretability	Low	**High**

5. Advantages & Disadvantages

Advantages

- **Simple** to understand and implement, with **no training time**.
- **Adapts instantly** to new data (just add points).
- Works for **multi-class** problems and **non-linear** boundaries.

Disadvantages

- **Slow predictions** and **large memory use** on big datasets.
- Suffers from the **curse of dimensionality** and needs feature scaling.
- **Sensitive to noisy and irrelevant** features.

6. Conclusion / Application

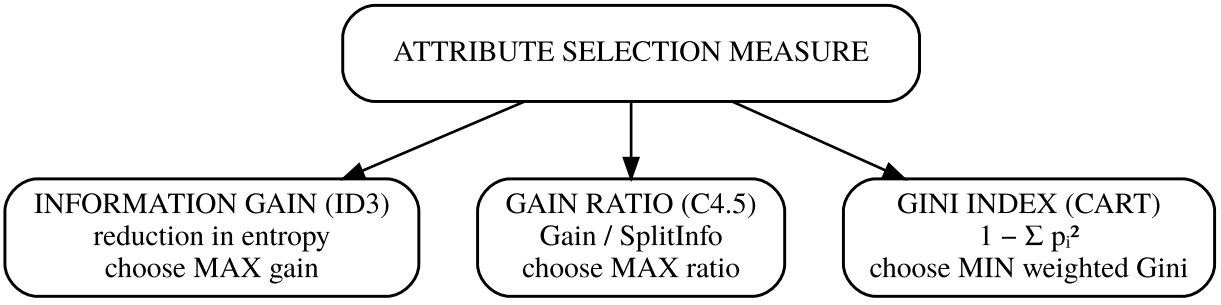
k-NN is used in **recommendation systems (similar users or products), handwriting and image recognition, medical diagnosis and anomaly detection**.

Q5. Compare Information Gain, Gain Ratio and Gini Index. (7 Marks)

1. Introduction / Definition

Information Gain, Gain Ratio and Gini Index are **attribute selection measures (splitting criteria)** used by decision-tree algorithms to choose the **best attribute** at each node. **Information Gain** (used by ID3) measures the **reduction in entropy**. **Gain Ratio** (used by C4.5) **normalizes Information Gain** to remove its bias toward multi-valued attributes. **Gini Index** (used by CART) measures the **probability of misclassifying** a randomly chosen element.

2. Block Diagram - Splitting Criteria and Formulas



Attribute selection measures and the algorithms that use them

$$\text{Gain}(S,A) = \text{Entropy}(S) - \sum_v \frac{|S_v|}{|S|}\text{Entropy}(S_v) \qquad \text{GainRatio}(S,A) = \frac{\text{Gain}(S,A)}{\text{SplitInfo}(S,A)}$$

$$\text{SplitInfo}(S,A) = -\sum_v \frac{|S_v|}{|S|}\log_2\frac{|S_v|}{|S|} \qquad \text{Gini}(S) = 1 - \sum_i p_i^2, \qquad \text{Gini}_A(S) = \sum_v \frac{|S_v|}{|S|}\text{Gini}(S_v)$$

3. Core Explanation / Working

(a) Information Gain (ID3)

- It measures **how much an attribute reduces uncertainty** (entropy) about the class.
- **Higher gain means a better attribute.**

- *Bias:* it **favours attributes with many distinct values** (for example “Day” or “Roll No.”). Such attributes create many tiny pure subsets, giving a high but meaningless gain.

(b) Gain Ratio (C4.5)

- It divides Information Gain by **SplitInfo**, the **intrinsic information** of the split, which is large when an attribute splits the data into many small partitions.
- This **penalizes multi-valued attributes** and removes the bias of Information Gain.
- *Issue:* when SplitInfo is close to 0, the ratio becomes unstable. C4.5 therefore only considers attributes with **above-average gain**.

(c) Gini Index (CART)

- It measures the **impurity** as the probability that a randomly chosen sample would be **misclassified** if it were labelled randomly according to the class distribution.
- Gini = 0 means pure. The maximum is 0.5 for 2 classes.
- **It uses no logarithms, so it is faster to compute.** CART produces **binary splits**.

(d) Worked comparison on the attribute “Outlook” (PlayTennis: S = [9+, 5−])

Information Gain

- Gain(S, Outlook) = 0.940 − 0.694 = **0.246** (computed in the ID3 question).

Gain Ratio

$$\text{SplitInfo}(S, \text{Outlook}) = -\frac{5}{14}\log_2\frac{5}{14} - \frac{4}{14}\log_2\frac{4}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.530 + 0.516 + 0.530 = 1.577$$

$$\text{GainRatio}(S, \text{Outlook}) = \frac{0.246}{1.577} = 0.156$$

Gini Index

$$\text{Gini}(S) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 1 - 0.413 - 0.128 = 0.459$$

$$\text{Gini}(\text{Sunny}) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.480, \qquad \text{Gini}(\text{Overcast}) = 1 - 1^2 = 0, \qquad \text{Gini}(\text{Rain}) = 0.480$$

$$\text{Gini}_{\text{Outlook}}(S) = \frac{5}{14}(0.480) + \frac{4}{14}(0) + \frac{5}{14}(0.480) = 0.343, \qquad \text{Gini gain} = 0.459 - 0.343 = 0.116$$

- All three measures **rank Outlook highest** for this dataset, so they agree here. They differ mainly on **many-valued attributes**.

4. Key Characteristics - Comparison Table

Basis	Information Gain	Gain Ratio	Gini Index
Used in	**ID3**	**C4.5**	**CART**
Based on	Entropy	Entropy + SplitInfo	Squared class probabilities
Formula	Entropy(S) − Σ (|Sv|/|S|) Entropy(Sv)	Gain / SplitInfo	1 − Σ p_i²
Selection rule	**Maximum** gain	**Maximum** ratio	**Minimum** weighted Gini
Range (2 classes)	0 to 1	0 to 1	0 to 0.5
Bias	Towards **many-valued** attributes	Towards **unbalanced** splits (low SplitInfo)	Towards **larger partitions**; isolates the most frequent class
Computation	Logarithms (slower)	Logarithms (slowest)	**No logarithms (fastest)**
Split type	Multi-way	Multi-way	**Binary**
Value for Outlook (PlayTennis)	0.246	0.156	Gini gain 0.116

5. Advantages & Disadvantages

Information Gain: simple and theoretically grounded, *but* biased toward many-valued attributes. *Gain Ratio:* **removes the multi-value bias**, *but* is unstable when SplitInfo is close to 0. *Gini Index:* **fast** and works well with binary splits and large data, *but* is less sensitive to small changes in class probabilities.

6. Conclusion / Application

The choice of criterion decides the algorithm. **Scikit-learn's DecisionTreeClassifier uses Gini by default** (CART), while **C4.5 and Weka's J48 use Gain Ratio**.

Unit 4: Artificial Neural Networks (ANN) & Deep Learning

Q1. Short Definitions (2 Marks Each)

(a) What is backpropagation?

- **Backpropagation (backward propagation of errors)** is a **supervised learning algorithm** for training **multilayer feed-forward neural networks**. It computes the **gradient of the error with respect to every weight**, using the **chain rule of calculus**, by passing the error **backward from the output layer to the input layer**. It then updates the weights with **gradient descent**:

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}, \qquad \Delta w_{ji} = \eta\, \delta_j\, x_{ji}$$

- It has two phases: a **forward pass** (compute the outputs) and a **backward pass** (compute the error terms δ and update the weights).

(b) What is the role of inductive bias in ANN?

- **Inductive bias** is the set of **assumptions** a learner uses to **generalize** beyond the training examples.

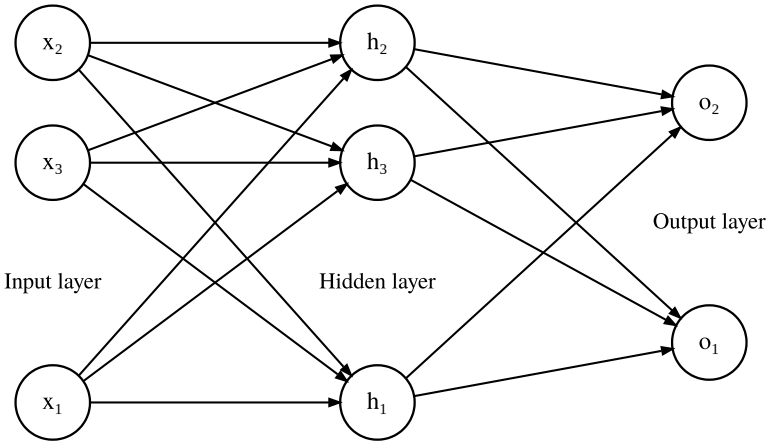
- The inductive bias of backpropagation ANNs is **"smooth interpolation between data points"**. Given two positive examples with no negative examples between them, the network tends to **label the points in between as positive** too.
- **Role:** it lets the network **generalize to unseen inputs**. The **architecture** (number of layers, units, connections, activation functions) also builds in bias. For example, **CNNs assume spatial locality and translation invariance**.

Q2. Derive the complete mathematical proof of the Backpropagation algorithm. (7 Marks)

1. Introduction / Definition

Backpropagation trains a multilayer network by **minimizing the squared error** between the target outputs t_k and the network outputs o_k, using **gradient descent** on the weights. The derivation uses the **chain rule** to find **∂E/∂w_ji** for weights going into **output units** and into **hidden units**, with sigmoid activation.

2. Block Diagram - Network and Notation



Forward pass computes the outputs; the backward pass sends the error terms δ from the output layer back to the hidden layer

Forward: $net_j = \sum_i w_{ji} x_{ji}, \quad o_j = \sigma\left(net_j\right) = \dfrac{1}{1 + e^{-net_j}}$ Backward: $\delta_k = o_k\left(1 - o_k\right)\left(t_k - o_k\right), \quad \delta_h = o_h$

Notation: x_{ji} = i-th input to unit j, w_{ji} = weight from unit i to unit j, o_j = output of unit j, t_j = target, η = learning rate, Downstream(j) = units that take the output of j as input.

3. Core Explanation / Working - The Derivation

Step 1 - Error function (for one training example d)

$$E_d(\mathbf{w}) = \frac{1}{2} \sum_{k \in \text{outputs}} \left(t_k - o_k\right)^2$$

Step 2 - Gradient descent rule

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

Step 3 - Apply the chain rule through net_j. The weight w_ji affects E only through net_j:

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \cdot x_{ji} \qquad \cdots (1)$$

$$\left(\text{since } net_j = \sum_i w_{ji} x_{ji}, \quad \frac{\partial net_j}{\partial w_{ji}} = x_{ji} \right)$$

- Now we need **∂E_d/∂net_j**. There are two cases.

Step 4 - Useful result: derivative of the sigmoid

$$\sigma(y) = \frac{1}{1 + e^{-y}}, \qquad \frac{d\sigma}{dy} = \frac{e^{-y}}{\left(1 + e^{-y}\right)^2} = \sigma(y)\,(1 - \sigma(y))$$

$$\Rightarrow \quad \frac{\partial o_j}{\partial net_j} = o_j\left(1 - o_j\right) \qquad \cdots (2)$$

CASE 1 - Training rule for OUTPUT unit weights

net_j affects E only through o_j:

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j} \cdot \frac{\partial o_j}{\partial net_j} \qquad \cdots (3)$$

First term:

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j}\left[\frac{1}{2}\sum_k \left(t_k - o_k\right)^2\right] = \frac{1}{2} \cdot 2\left(t_j - o_j\right) \cdot \frac{\partial\left(t_j - o_j\right)}{\partial o_j} = -\left(t_j - o_j\right) \qquad \cdots (4)$$

(only the term with k = j depends on o_j)

Substituting (2) and (4) into (3):

$$\frac{\partial E_d}{\partial net_j} = -\left(t_j - o_j\right) o_j\left(1 - o_j\right)$$

Define the **error term** δ_j = −∂E_d/∂net_j:

$$\delta_j = o_j\left(1 - o_j\right)\left(t_j - o_j\right), \qquad \Delta w_{ji} = \eta\, \delta_j\, x_{ji} \qquad \text{(OUTPUT unit)}$$

CASE 2 - Training rule for HIDDEN unit weights

A hidden unit j affects E only through the units **k ∈ Downstream(j)** that it feeds:

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \,\in\, \text{Downstream}(j)} \frac{\partial E_d}{\partial net_k} \cdot \frac{\partial net_k}{\partial net_j} = \sum_{k \,\in\, \text{Downstream}(j)} (-\delta_k) \cdot \frac{\partial net_k}{\partial o_j} \cdot \frac{\partial o_j}{\partial net_j}$$

Since $net_k = \sum_j w_{kj} o_j$, we get $\frac{\partial net_k}{\partial o_j} = w_{kj}$, and from (2) $\frac{\partial o_j}{\partial net_j} = o_j\left(1 - o_j\right)$:

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \,\in\, \text{Downstream}(j)} (-\delta_k)\, w_{kj}\, o_j\left(1 - o_j\right)$$

So δ_j = −∂E_d/∂net_j gives:

$$\delta_h = o_h(1 - o_h) \sum_{k \in \text{Downstream}(h)} w_{kh}\,\delta_k, \qquad \Delta w_{hi} = \eta\,\delta_h\,x_{hi} \qquad \text{(HIDDEN unit)}$$

- The hidden unit's error is the **weighted sum of the downstream error terms**. This is the error **propagated backward**, which gives the algorithm its name.

Step 5 - The complete Backpropagation algorithm (stochastic gradient descent)

```
BACKPROPAGATION(training_examples, η, n_in, n_out, n_hidden)
1. Initialize all weights to small random numbers (for example -0.05 to +0.05)
2. Until the termination condition is met, for each <x, t> in training_examples:
   (a) FORWARD:  propagate x through the network; compute o_u for every unit u
   (b) For each OUTPUT unit k:   δ_k = o_k (1 - o_k)(t_k - o_k)
   (c) For each HIDDEN unit h:   δ_h = o_h (1 - o_h) Σ w_kh δ_k
   (d) UPDATE every weight:      w_ji = w_ji + η δ_j x_ji
```

Optional - Momentum term: `Δw_ji(n) = η δ_j x_ji + α Δw_ji(n−1)`. This speeds up convergence and helps escape small local minima.

4. Key Characteristics - Output vs. Hidden Unit Rule

Basis	Output Unit k	Hidden Unit h
Error term δ	o_k(1 − o_k)(t_k − o_k)	o_h(1 − o_h) Σ w_kh δ_k
Target known?	**Yes** (t_k)	**No**, so the error is borrowed from downstream units
Factor o(1 − o)	Derivative of the sigmoid	Derivative of the sigmoid
Weight update	Δw_kh = η δ_k o_h	Δw_hi = η δ_h x_i
Computed	First (backward pass starts here)	After the output δ values

5. Advantages & Disadvantages

Advantages

- Efficiently computes the gradients for **all weights in one backward pass**, in O(W) time.
- Can learn **complex non-linear** mappings (it is a universal approximator).
- It is the **foundation of all modern deep learning** training.

Disadvantages

- Can converge to **local minima**. Training can be **slow**.
- The **vanishing gradient problem** affects sigmoid networks: the factor o(1 − o) ≤ 0.25 shrinks the gradient in deep layers.
- **Sensitive** to the learning rate, weight initialization and overfitting.

6. Conclusion / Application

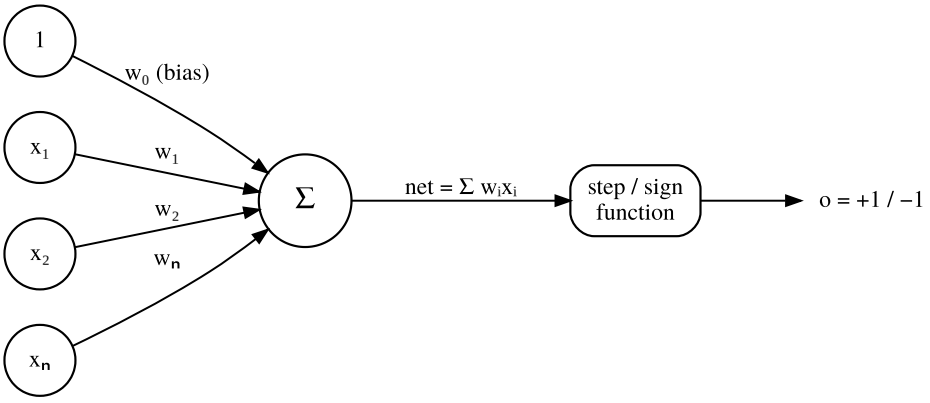
Backpropagation trains networks used in **image recognition, speech recognition, machine translation and large language models**.

Q3. Differentiate between a single-layer perceptron and a multilayer perceptron with block diagrams. (7 Marks)

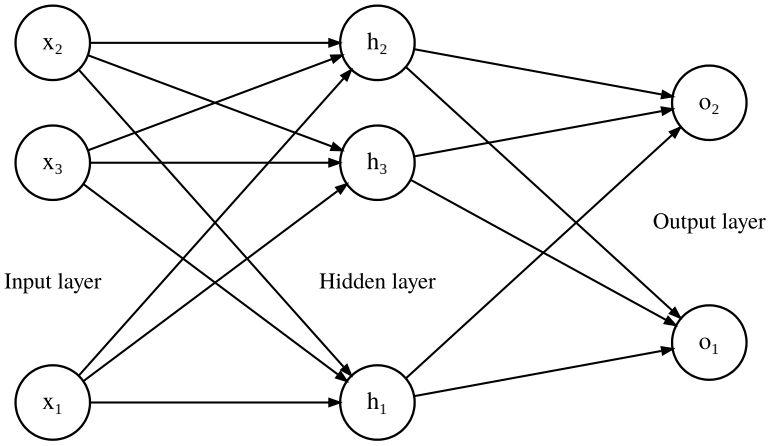
1. Introduction / Definition

A **Single-Layer Perceptron (SLP)** is the simplest neural network, proposed by **Rosenblatt (1958)**. It has **one layer of weights** connecting the inputs directly to an output unit with a **step (threshold) activation**, and it can only learn **linearly separable** functions. A **Multilayer Perceptron (MLP)** has **one or more hidden layers** with **non-linear activations**. It is trained with **backpropagation** and can learn **non-linear** functions such as XOR.

2. Block Diagrams



Single-layer perceptron: weighted sum followed by a step function



Multilayer perceptron with one hidden layer (sigmoid / ReLU units), trained by backpropagation

3. Core Explanation / Working

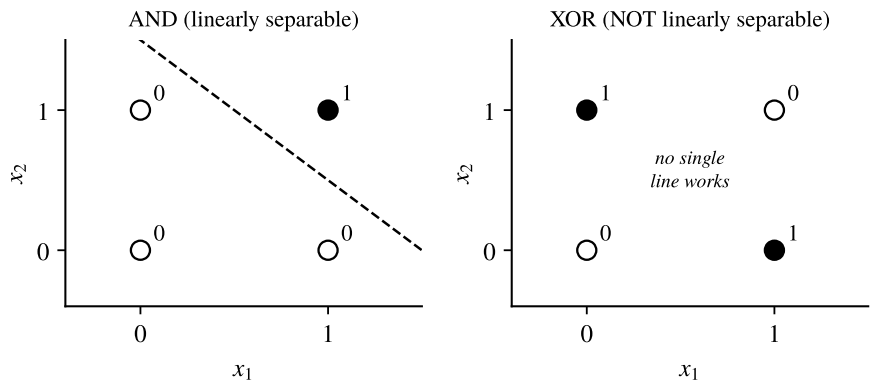
(a) Single-Layer Perceptron

- **Output:** `o = sign(Σ wi·xi)`, where `x0 = 1` is the bias input.
- **Perceptron training rule:**

$$w_i \leftarrow w_i + \Delta w_i, \qquad \Delta w_i = \eta\ (t - o)\ x_i$$

- The weights change **only when the output is wrong** (t ≠ o).
- **Perceptron Convergence Theorem:** if the data is **linearly separable**, the rule converges in a finite number of steps.

- **Limitation (Minsky and Papert, 1969):** it **cannot learn XOR**, because XOR is **not linearly separable**.



AND is linearly separable (one line separates the classes); XOR is not, so a single-layer perceptron cannot learn it

(b) Multilayer Perceptron

- Layers: an **input layer**, then **one or more hidden layers**, then an **output layer**. Each layer is fully connected to the next.
- **Activations:** non-linear and **differentiable** (sigmoid, tanh, ReLU), so gradients can be computed.
- **Training: backpropagation** with gradient descent minimizes $E = \frac{1}{2} \Sigma (t_k - o_k)^2$.
- **Solves XOR:** the hidden units create **new feature representations**. For example h1 = OR(x1, x2) and h2 = AND(x1, x2), then output = h1 AND NOT h2.
- **Universal Approximation Theorem:** an MLP with **one hidden layer** and enough units can approximate **any continuous function**.

4. Key Characteristics - SLP vs. MLP

Basis	Single-Layer Perceptron	Multilayer Perceptron
Layers	Input + output (**no hidden layer**)	Input + **one or more hidden layers** + output
Activation function	**Step / sign** (non-differentiable)	**Sigmoid, tanh, ReLU** (differentiable)
Learning rule	Perceptron rule $\Delta w = \eta(t - o)x$	**Backpropagation** (gradient descent)
Decision boundary	**Linear** (a hyperplane)	**Non-linear** (arbitrary shapes)
Can solve XOR?	**No**	**Yes**
Problems solved	Linearly separable (AND, OR)	Linear and non-linear (XOR, image recognition)
Convergence	Guaranteed if linearly separable	May reach a local minimum
Complexity	Simple, fast	More parameters, slower training
Output	Binary class	Multi-class or continuous

5. Advantages & Disadvantages

SLP

- *Advantages:* **simple and fast**; guaranteed convergence on separable data.
- *Disadvantages:* **only linear boundaries**, so it cannot solve XOR.

MLP

- *Advantages:* learns **complex non-linear** patterns; universal approximator.
- *Disadvantages:* **slower training**, risk of **overfitting** and **local minima**, and it is a **black box** (hard to interpret).

6. Conclusion / Application

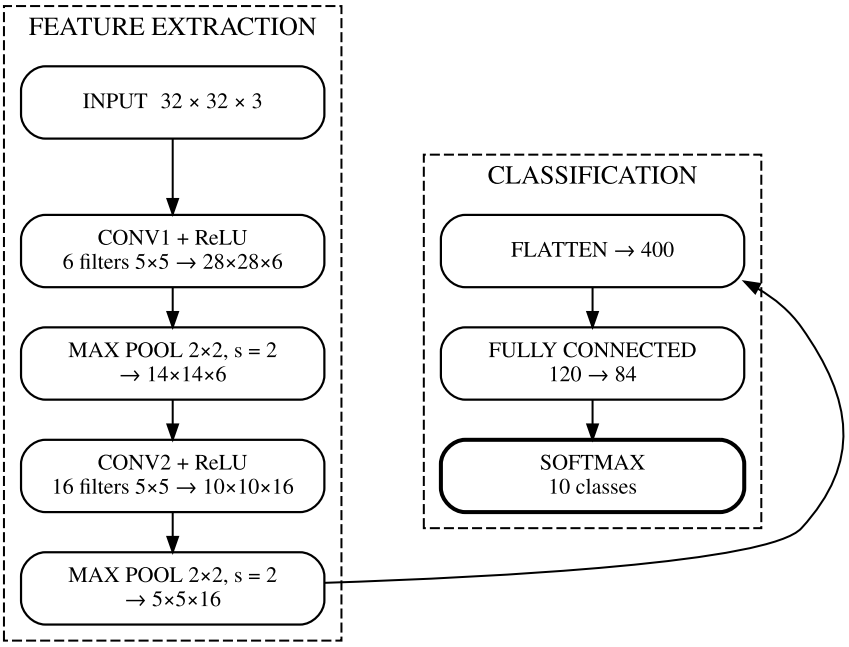
SLPs are used for **simple linear classification**, while MLPs power **credit-risk models, speech recognition and the fully connected layers of deep networks**.

Q4. Explain the layered architecture (convolution, pooling, fully connected) of a Convolutional Neural Network (CNN). (7 Marks)

1. Introduction / Definition

A **Convolutional Neural Network (CNN)** is a **deep learning** architecture designed for **grid-like data such as images**. It automatically learns **spatial feature hierarchies** (edges, then shapes, then objects) using three main types of layers: **convolution layers** (feature extraction), **pooling layers** (down-sampling) and **fully connected layers** (classification).

2. Block Diagram - CNN Architecture



CNN architecture (LeNet-style): feature extraction (left), then classification (right)

3. Core Explanation / Working

(a) Input Layer

- Holds the raw image pixels as a tensor of **Height × Width × Channels** (for example 32 × 32 × 3 for an RGB image).

(b) Convolution Layer (feature extraction)

- A small **filter (kernel)**, for example 3 × 3 or 5 × 5, **slides over the input**. At each position it computes the **dot product** of the filter weights and the input patch, which produces a **feature map**.
- **Key ideas:**
  - **Local connectivity:** each neuron sees only a small region (its **receptive field**).
  - **Parameter (weight) sharing:** the same filter is used across the whole image, so there are **far fewer parameters** than in a fully connected layer.
  - Each filter detects one feature (an edge, a curve, a colour blob).
- **Hyper-parameters:** number of filters (K), filter size (F), **stride (S)**, **padding (P)**.
- **Output size:**

$$\text{Output size} = \frac{W - F + 2P}{S} + 1 \qquad \text{e.g. } W = 32,\ F = 5,\ P = 0,\ S = 1:\ \ \frac{32 - 5 + 0}{1} + 1 = 28 \ \Rightarrow\ 28 \times 28 \times$$

- *Convolution example (3 × 3 input, 2 × 2 kernel, stride 1):*

$$\underbrace{\begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}}_{\text{Input } I} * \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}}_{\text{Kernel } K} = \underbrace{\begin{bmatrix} 2 & 5 \\ 1 & 1 \end{bmatrix}}_{\text{Feature map}}$$

- Top-left: $1(1) + 2(0) + 0(0) + 1(1) = 2$. Top-right: $2(1) + 0(0) + 1(0) + 3(1) = 5$.
- Bottom-left: $0(1) + 1(0) + 2(0) + 1(1) = 1$. Bottom-right: $1(1) + 3(0) + 1(0) + 0(1) = 1$.

(c) Activation Layer (ReLU)

- **ReLU(x) = max(0, x)** is applied element-wise. It adds **non-linearity** and avoids the vanishing gradient problem of sigmoid.

(d) Pooling Layer (down-sampling)

- It **reduces the spatial size** (height and width) of the feature maps while **keeping the important information**. This reduces computation and **controls overfitting**.
- It gives **translation invariance**: small shifts in the input do not change the output.
- It has **no learnable parameters**.
- **Max pooling** keeps the maximum value in each window. **Average pooling** keeps the mean.

$$\underbrace{\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 9 & 1 \\ 3 & 4 & 0 & 8 \end{bmatrix}}_{\text{Input } 4 \times 4} \xrightarrow{\text{max pool } 2 \times 2,\ s = 2} \begin{bmatrix} 6 & 4 \\ 7 & 9 \end{bmatrix} \qquad \xrightarrow{\text{avg pool}} \begin{bmatrix} 3.75 & 2.25 \\ 4.00 & 4.50 \end{bmatrix}$$

(e) Flatten Layer

- Converts the final 3-D feature maps into a **1-D vector** (for example 5 × 5 × 16 = 400 values) for the dense layers.

(f) Fully Connected (Dense) Layer (classification)

- Every neuron connects to **every neuron** in the previous layer, as in an MLP.

- It **combines the extracted high-level features** to make the final decision.
- **Dropout** is often added to prevent overfitting.

(g) Output Layer (Softmax)

- **Softmax** converts the scores into **class probabilities** that sum to 1:

$$P(\text{class}_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Training uses **cross-entropy loss** with **backpropagation** and gradient descent (Adam or SGD).

4. Key Characteristics - Comparison of CNN Layers

Basis	Convolution Layer	Pooling Layer	Fully Connected Layer
Purpose	**Feature extraction**	**Down-sampling /** dimensionality reduction	**Classification**
Learnable parameters	**Yes** (filter weights + bias)	**No**	**Yes** (most parameters are here)
Connectivity	Local (receptive field)	Local window	**Global** (all-to-all)
Effect on size	Same or slightly smaller (depends on padding)	Halves H and W (2 × 2, stride 2)	Outputs a fixed vector
Key property	Weight sharing	Translation invariance	Combines features
Example operation	5 × 5 filter, stride 1	2 × 2 max pool	400 → 120 → 84 → 10

5. Advantages & Disadvantages

Advantages

- **Automatic feature learning**, with no hand-crafted features.
- **Weight sharing and pooling** give **far fewer parameters** than an MLP on raw pixels.
- **Translation invariant**, and state-of-the-art accuracy on images and video.

Disadvantages

- Needs **large labelled datasets** and **GPU power** for training.
- **Black box**, so it is hard to interpret.
- Pooling **loses exact spatial information**, and the network is sensitive to rotation and scale unless trained with augmentation.

6. Conclusion / Application

CNNs (LeNet-5, AlexNet, VGG, ResNet) power **face recognition, medical image diagnosis (X-ray and MRI), self-driving car vision and object detection (YOLO)**.

Unit 5: Reinforcement Learning & Genetic Algorithms

Q1. Short Definitions (2 Marks Each)

(a) What is Q-learning?

- **Q-learning** is a **model-free, off-policy reinforcement learning** algorithm. It learns the **action-value function Q(s, a)**, which is the expected cumulative reward of taking action a in state s and then acting optimally. It **does not need a model** of the environment's transitions or rewards.
- **Update rule (Bellman equation):**

$$Q(s,a) \leftarrow Q(s,a) + \alpha[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a) \,]$$

where α = learning rate, γ = discount factor ($0 \le \gamma < 1$), r = reward and s' = next state.

- The optimal policy is then $\pi^*(s) = \arg\max_a Q(s,a)$. Exploration is usually handled with an **ε-greedy** strategy.

(b) What is the crowding problem in Genetic Algorithms?

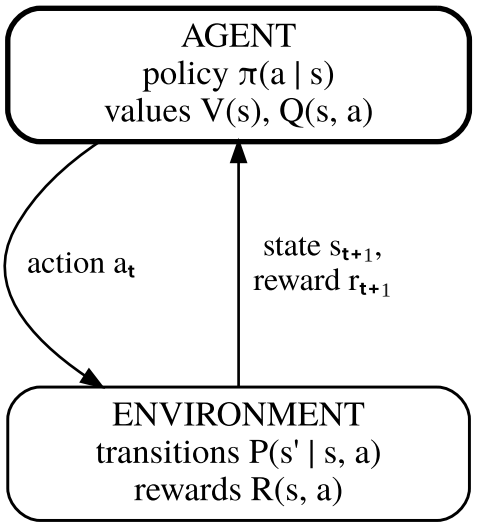
- **Crowding** happens when **one highly fit individual (and its near-identical offspring) quickly reproduces and takes over a large part of the population**.
- This **reduces diversity**, so the GA **converges prematurely to a local optimum** and stops exploring other regions of the search space.
- **Remedies: fitness sharing** (lower the fitness of similar individuals), **crowding replacement** (a new child replaces the most similar parent), **tournament or rank selection** instead of fitness-proportionate selection, and **restricted mating** between similar individuals.

Q2. Explain Reinforcement Learning through the Markov Decision Process (MDP). (7 Marks)

1. Introduction / Definition

Reinforcement Learning (RL) is a learning paradigm in which an **agent** learns to make a **sequence of decisions** by interacting with an **environment**, so as to **maximize the cumulative reward**. The RL problem is formally modelled as a **Markov Decision Process (MDP)**, a mathematical framework for sequential decision-making under uncertainty.

2. Block Diagram - Agent-Environment Interaction



Agent-environment interaction in an MDP

Trajectory: $s_0 \overset{a_0}{\to} (r_1)\, s_1 \overset{a_1}{\to} (r_2)\, s_2 \cdots$ Return: $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$

3. Core Explanation / Working

(a) Components of an MDP: the tuple (S, A, P, R, γ)

- **S - States:** all possible situations of the environment (for example positions on a grid).
- **A - Actions:** the choices available to the agent (for example Up, Down, Left, Right).
- **P - Transition probability:** `P(s' | s, a)`, the probability of reaching state s' after taking action a in state s.
- **R - Reward function:** `R(s, a)` or `R(s, a, s')`, the immediate numeric feedback.
- **γ - Discount factor (0 ≤ γ < 1):** how much future rewards are worth compared with immediate ones.

(b) The Markov Property

- *"The future depends only on the present state, not on the past."*

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0) = P(s_{t+1} \mid s_t, a_t)$$

(c) Policy π

- A **mapping from states to actions**: `π(s) = a` (deterministic) or `π(a | s)` (stochastic).
- **Goal of RL:** find the **optimal policy π*** that maximizes the expected return.

(d) Value Functions

- **State-value function:** `V^π(s) = E_π [ Σ γ^k r_t+k+1 | s_t = s ]`, the expected return starting from s and following π.
- **Action-value function:** `Q^π(s, a) = E_π [ Σ γ^k r_t+k+1 | s_t = s, a_t = a ]`

(e) Bellman Optimality Equations

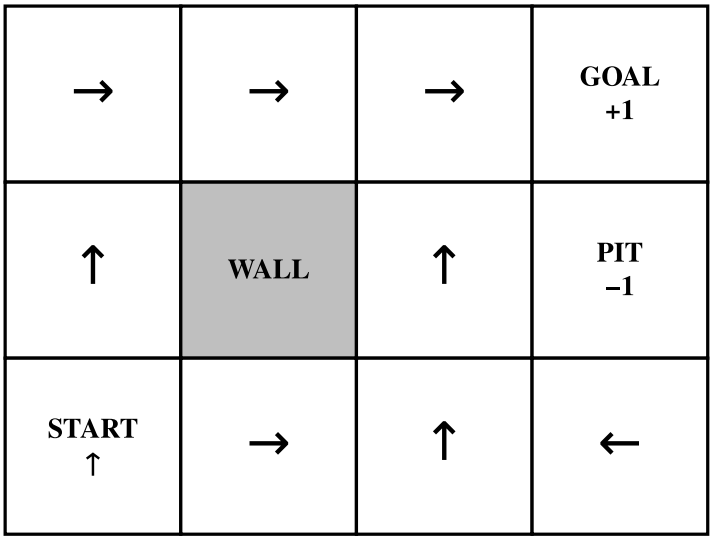
$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a)\,[R(s, a, s') + \gamma V^*(s')]$$

$$Q^*(s,a) = \sum_{s'} P(s' \mid s,a)\,[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')], \qquad \pi^*(s) = \arg\max_a Q^*(s,a)$$

(f) Solving an MDP

- **Model known (P and R given): Dynamic Programming.**
  - **Value Iteration:** repeatedly apply the Bellman optimality update until V converges.
  - **Policy Iteration:** alternate between **policy evaluation** and **policy improvement**.
- **Model unknown: model-free RL.**
  - **Q-learning** (off-policy): `Q(s,a) ← Q(s,a) + α[r + γ max Q(s',a') − Q(s,a)]`
  - **SARSA** (on-policy): uses the action a' actually taken.
  - **Exploration vs. exploitation: ε-greedy** takes a random action with probability ε and the best known action otherwise.

(g) Example - Grid World MDP



Grid-world MDP ($\gamma = 0.9$; +1 at GOAL, −1 at PIT, −0.04 per step; 0.8 chance of the intended move). Arrows show the optimal policy π^*

4. Key Characteristics - Reinforcement Learning vs. Supervised Learning

Basis	Reinforcement Learning (MDP)	Supervised Learning
Feedback	**Scalar reward**, often delayed	**Correct label** for every input
Data	Generated by **interaction**	Fixed labelled dataset
Decisions	**Sequential**: actions affect future states	Independent predictions
Goal	Maximize **cumulative reward**	Minimize prediction error
Exploration	Required (explore vs. exploit)	Not needed
Example	Game playing, robot control	Image classification

5. Advantages & Disadvantages

Advantages

- Learns **optimal long-term behaviour** without labelled data.

- The MDP gives a **rigorous mathematical framework** (Bellman equations, convergence guarantees).
- Handles **uncertainty** in action outcomes through transition probabilities.

Disadvantages

- **Curse of dimensionality:** large state and action spaces make tabular methods impractical, so function approximation (Deep RL) is needed.
- Needs **many interactions** (sample-inefficient), and **reward design** is difficult.
- Real problems are often **partially observable** (POMDPs), which breaks the Markov assumption.

6. Conclusion / Application

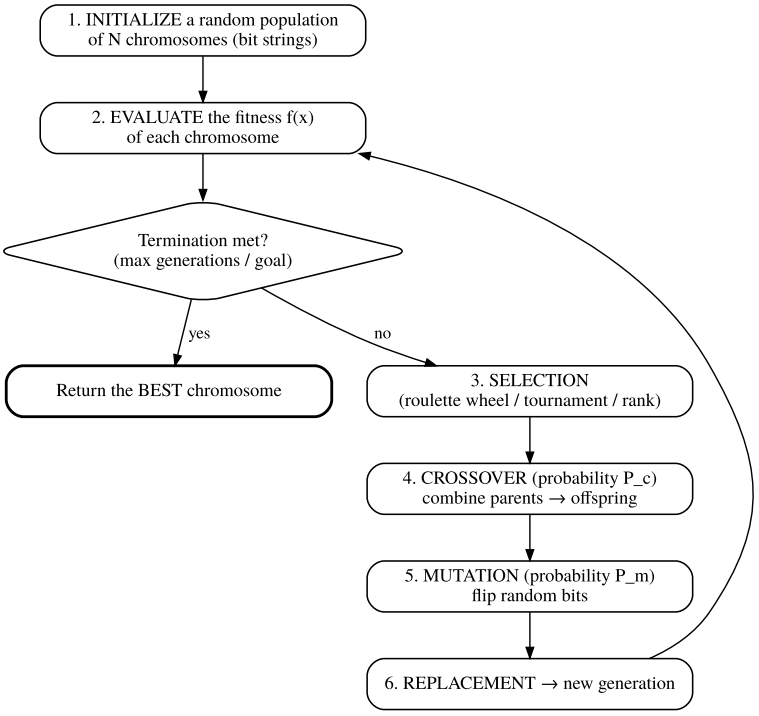
MDP-based RL powers **AlphaGo, robotics control, autonomous driving, inventory and resource management, and RLHF tuning of large language models**.

Q3. Trace the Genetic Algorithm cycle, detailing the roles of fitness functions, crossover and mutation. (7 Marks)

1. Introduction / Definition

A **Genetic Algorithm (GA)** is a **search and optimization technique** inspired by **Darwin's natural selection and genetics**. It keeps a **population of candidate solutions (chromosomes)**. Each generation it **selects** the fitter individuals, **recombines** them with **crossover**, and **mutates** them, so that solutions evolve towards better fitness.

2. Flowchart - GA Cycle



Genetic Algorithm cycle

3. Core Explanation / Working

(a) Encoding

- Each candidate solution is encoded as a **chromosome**, usually a **binary string**. Each bit is a **gene**.

(b) Fitness Function: role

- It **measures how good** a chromosome is at solving the problem. It is the GA's **only link to the problem**.
- It decides **which individuals survive and reproduce**: higher fitness means a higher chance of selection.
- A good fitness function is **fast to compute** and **clearly separates** good solutions from bad ones.

(c) Selection

- **Roulette wheel (fitness-proportionate):** `P(select i) = f(i) / Σ f(j)`.
- **Tournament:** pick k individuals at random and choose the best of them.
- **Elitism:** copy the best individual(s) directly into the next generation.

(d) Crossover: role

- **Exploitation:** it **combines good building blocks (schemata)** from two parents to make potentially better offspring.
- Types:

Type	Parents	Offspring
Single-point (after bit 3)	P1 = 101 | 10, P2 = 010 | 01	C1 = 101 | **01**, C2 = 010 | **10**
Two-point (bits 3-5 swapped)	P1 = 11 | 001 | 1, P2 = 00 | 110 | 0	C1 = 11 | **110** | 1, C2 = 00 | **001** | 0
Uniform	each bit is taken from either parent at random (using a mask)	-

(e) Mutation: role

- **Exploration:** it **flips random bits** with a small probability Pm (for example 0.001 to 0.01).
- It **maintains diversity** and lets the GA **recover lost genetic material**, which prevents premature convergence (**crowding**).

	Chromosome
Before mutation	1 0 **1** 1 0 1
After flipping bit 3	1 0 **0** 1 0 1

(f) Worked Trace: maximize **f(x) = x²** for x in 0 to 31 (5-bit chromosomes), N = 4

Generation 0 - Evaluation and Selection

No.	Chromosome	x	Fitness f(x) = x^2	P(select) = f / Σf	Expected count (P × 4)	Actual count (roulette)
1	01101	13	169	0.14	0.58	1
2	11000	24	576	0.49	1.97	2
3	01000	8	64	0.06	0.22	0
4	10011	19	361	0.31	1.23	1
			Σ = 1170, Avg = 293, **Max = 576**			

Crossover (single-point; the crossover point is chosen at random)

Pair	Parents	Cross point	Offspring	x	Fitness x^2
1	0110 | 1 and 1100 | 0	4	**01100** and **11001**	12, 25	144, 625
2	11 | 000 and 10 | 011	2	**11011** and **10000**	27, 16	729, 256

Mutation (Pm = 0.001: for example, no bit flips in this generation. If bit 3 of 10000 were flipped, it would become 10100 = 20, with fitness 400.)

Generation 1 - Result

Chromosome	x	Fitness
01100	12	144
11001	25	625
11011	27	**729**
10000	16	256
		Σ = 1754, **Avg = 439**, **Max = 729**

- **In one generation, the average fitness rose from 293 to 439 and the maximum from 576 to 729.** Repeating the cycle converges toward **11111 (x = 31, f = 961)**.

4. Key Characteristics - Roles of the GA Operators

Operator	Role	Search Effect	Typical Rate
Fitness function	Evaluates solution quality	Guides the direction of the search	-
Selection	Picks parents by fitness	**Exploitation** (survival of the fittest)	-
Crossover	Recombines parent genes	**Exploitation** of good building blocks	Pc = 0.6 to 0.9
Mutation	Randomly alters genes	**Exploration**, keeps diversity	Pm = 0.001 to 0.01
Elitism	Preserves the best individuals	Guarantees no loss of the best solution	1 or 2 individuals

5. Advantages & Disadvantages

Advantages

- **Global search** that is less likely to get stuck in local optima than gradient methods.
- Works on **non-differentiable, discontinuous or noisy** objective functions. **No gradient** is needed.
- **Naturally parallel**, and suits **complex combinatorial** problems (scheduling, TSP).

Disadvantages

- **Computationally expensive**, because of many fitness evaluations.
- **No guarantee** of finding the global optimum. Results vary from run to run.
- Parameters (N, Pc, Pm) need **tuning**, and **crowding / premature convergence** can occur.

6. Conclusion / Application

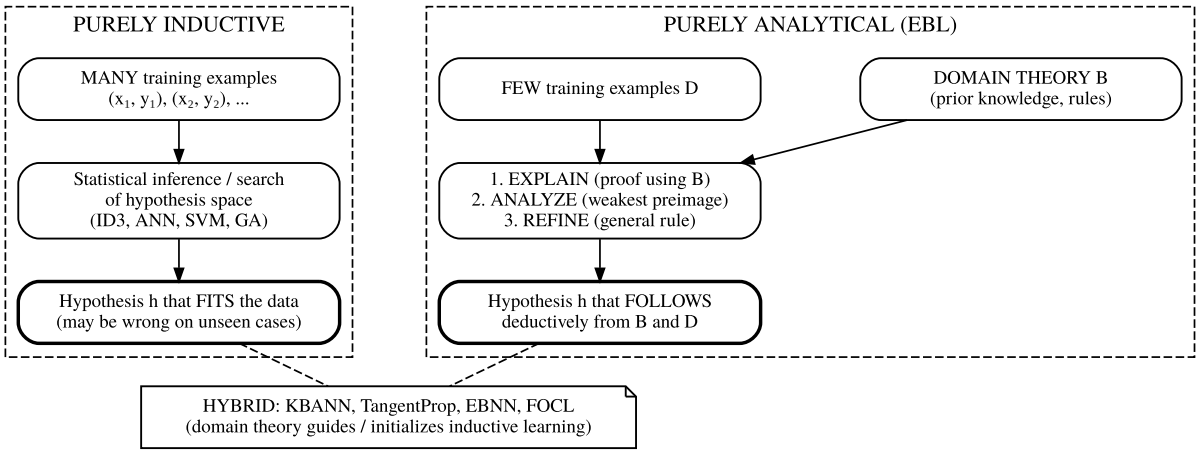
GAs are used in **timetable and job-shop scheduling, the travelling salesman problem, neural-network architecture search, antenna design (NASA) and feature selection** in ML.

Q4. Compare purely analytical learning with purely inductive learning. (7 Marks)

1. Introduction / Definition

Inductive learning forms **general hypotheses from many specific training examples** by finding **statistical regularities** in the data, with no prior domain knowledge. **Analytical learning** (for example **Explanation-Based Learning, EBL**) uses **prior domain knowledge (a domain theory)** together with **very few examples**. It **deductively explains** why an example is an instance of the target concept and generalizes from that explanation.

2. Block Diagram



Purely inductive vs. purely analytical learning

3. Core Explanation / Working

(a) Purely Inductive Learning

- **Given:** a hypothesis space H and training examples D.
- **Find:** a hypothesis h ∈ H that is **consistent with D**.

- **Inference type: induction**, from specific cases to a general rule. It is **statistical** and not guaranteed to be correct.
- **Needs a large number of examples** to generalize reliably, because of the sample complexity of PAC learning.
- **Justification:** the hypothesis is **statistically** justified by the data.
- *Examples:* Decision Trees (ID3), Neural Networks, SVM, Candidate Elimination, Genetic Algorithms.

(b) Purely Analytical Learning (Explanation-Based Learning)

- **Given:** H, training examples D, **and a domain theory B** that explains the training examples.
- **Find:** a hypothesis h consistent with both D and B.
- **Inference type: deduction**. The hypothesis is **logically entailed** by B together with the examples.
- **Steps of Prolog-EBG:**
    1. **Explain:** build a **proof (explanation)** of how the training example satisfies the target concept, using the domain theory.
    2. **Analyze:** compute the **weakest preimage** of the target concept with respect to the explanation, so that **only the relevant features** are kept.
    3. **Refine:** add the new general **Horn clause rule** to the hypothesis.
- **Can learn from even a single example**, because the knowledge comes from B.
- *Example:* learning **"SafeToStack(x, y)"** from one example, using a domain theory about weights, volumes and densities. Similarly, a chess program learns a general rule like "a fork wins material" from a single game.

4. Key Characteristics - Comparison Table

Basis	Purely Inductive Learning	Purely Analytical Learning
Input	**Many training examples**	**Domain theory + few examples**
Prior knowledge	**Not needed**	**Essential** (a correct, complete domain theory)
Reasoning type	**Inductive** (specific to general)	**Deductive** (logical proof)
Goal	Hypothesis that **fits the data**	Hypothesis that **follows from the domain theory**
Justification	Statistical	Logical
Number of examples needed	Large	Very few (even **one**)
Effect of noisy data	Can tolerate some noise	Not affected (relies on theory)
Main pitfall	**Insufficient data** gives poor generalization	**Imperfect domain theory** gives wrong hypotheses
What is learned	New knowledge **not** in prior knowledge	**Reformulation** of existing knowledge (speed-up learning)
Examples	ID3, ANN, SVM, k-NN, GA	**Explanation-Based Learning (Prolog-EBG)**

5. Advantages & Disadvantages

Inductive Learning

- *Advantages:* needs **no domain expertise**; can discover **genuinely new patterns**.
- *Disadvantages:* needs **large datasets**; generalization may be **wrong** on unseen data.

Analytical Learning

- *Advantages:* learns from **very few examples**; hypotheses are **logically justified** and explainable.
- *Disadvantages:* needs a **complete and correct domain theory**, which is rarely available; it **cannot learn anything beyond** what B already implies.

Hybrid approach: **KBANN** (Knowledge-Based ANN) builds an initial network from the domain theory and then **refines it inductively** with backpropagation, combining the strengths of both.

6. Conclusion / Application

Inductive learning drives **data-rich applications such as image recognition and recommendation**, while analytical learning is used in **expert systems, planning (SOAR, PRODIGY) and theorem proving**, where strong domain knowledge exists.

Numerical Problems (PYQ Templates)

These five types of numericals come up again and again in AKTU BCS-055 papers. Each is solved the way you should write it in the exam: formula first, then substitution, then the answer.

N1. Decision Tree (ID3): find the root node using Entropy and Information Gain. (7 Marks)

Given: the PlayTennis dataset of 14 days (9 Yes, 5 No). The full table is in Unit 3, Q2.

Formulas used

$$\text{Entropy}(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Step 1: Entropy of the whole dataset, S = [9+, 5−]

$$\text{Entropy}(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.410 + 0.530 = 0.940$$

Step 2: Gain of Outlook. Sunny = [2+, 3−], Overcast = [4+, 0−], Rain = [3+, 2−]

$$\text{Entropy}(S_{\text{Sunny}}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.971$$

$$\text{Entropy}(S_{\text{Overcast}}) = 0 \quad (\text{pure}), \qquad \text{Entropy}(S_{\text{Rain}}) = 0.971$$

$$\text{Gain}(S, \text{Outlook}) = 0.940 - \left[\frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971)\right] = 0.940 - 0.694 = 0.246$$

Step 3: Gain of Humidity. High = [3+, 4−], Normal = [6+, 1−]

$$\text{Gain}(S, \text{Humidity}) = 0.940 - \left[\frac{7}{14}(0.985) + \frac{7}{14}(0.592)\right] = 0.151$$

Step 4: Gain of Wind. Weak = [6+, 2−], Strong = [3+, 3−]

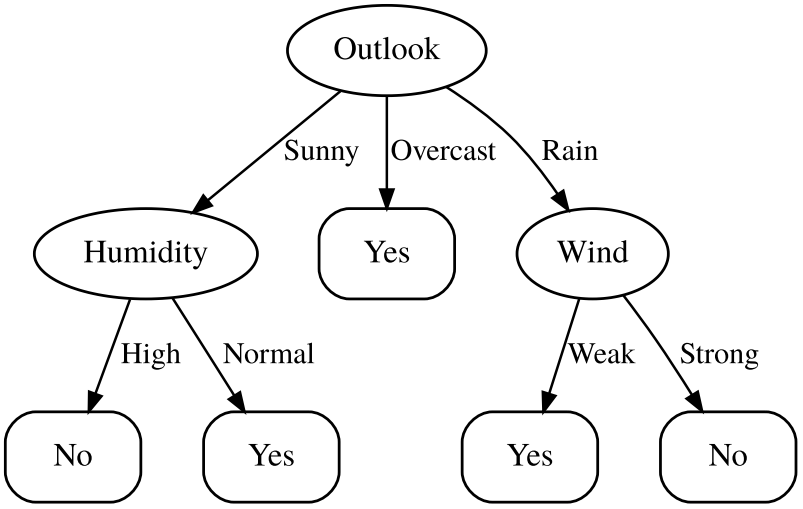
$$\text{Gain}\,(S, \text{Wind}) = 0.940 - \left[\frac{8}{14}(0.811) + \frac{6}{14}(1.000)\right] = 0.048$$

Step 5: Gain of Temperature. Hot = [2+, 2−], Mild = [4+, 2−], Cool = [3+, 1−]

$$\text{Gain}\,(S, \text{Temp}) = 0.940 - \left[\frac{4}{14}(1.0) + \frac{6}{14}(0.918) + \frac{4}{14}(0.811)\right] = 0.029$$

Attribute	Outlook	Humidity	Wind	Temperature
Information Gain	**0.246**	0.151	0.048	0.029

Answer: Outlook has the highest gain, so **Outlook is the root node**. Repeating the same steps on each branch gives Humidity under Sunny and Wind under Rain (worked in full in Unit 3, Q2).



Final ID3 decision tree

N2. Naive Bayes: classify a new tuple (Confident, Studied, Sick → Result). (7 Marks)

Given training data:

No.	Confident	Studied	Sick	Result
1	Yes	No	No	Fail
2	Yes	No	Yes	Pass
3	No	Yes	Yes	Fail
4	No	Yes	No	Pass
5	Yes	Yes	Yes	Pass

Classify: X = (Confident = Yes, Studied = Yes, Sick = No)

Formula used (Naive Bayes rule)

$$P(C \mid X) \;\propto\; P(C)\,\prod_i P(x_i \mid C), \qquad \text{Predicted class} = \arg\max_C \; P(C)\,\prod_i P(x_i \mid C)$$

Step 1: Prior probabilities. There are 3 Pass and 2 Fail.

$$P(\text{Pass}) = \frac{3}{5} = 0.6, \qquad P(\text{Fail}) = \frac{2}{5} = 0.4$$

Step 2: Likelihoods. Count within each class.

Attribute value	Pass (out of 3)	Fail (out of 2)
Confident = Yes	rows 2, 5 $\rightarrow \frac{2}{3}$	row 1 $\rightarrow \frac{1}{2}$
Studied = Yes	rows 4, 5 $\rightarrow \frac{2}{3}$	row 3 $\rightarrow \frac{1}{2}$
Sick = No	row 4 $\rightarrow \frac{1}{3}$	row 1 $\rightarrow \frac{1}{2}$

Step 3: Multiply prior × likelihoods

$$P(X \mid \text{Pass})\,P(\text{Pass}) = \frac{2}{3}\times\frac{2}{3}\times\frac{1}{3}\times\frac{3}{5} = \frac{4}{45} = 0.0889$$

$$P(X \mid \text{Fail})\,P(\text{Fail}) = \frac{1}{2}\times\frac{1}{2}\times\frac{1}{2}\times\frac{2}{5} = \frac{1}{20} = 0.0500$$

Step 4: Posterior probabilities (divide each by the sum 0.0889 + 0.0500 = 0.1389)

$$P(\text{Pass} \mid X) = \frac{0.0889}{0.1389} = 0.64, \qquad P(\text{Fail} \mid X) = \frac{0.0500}{0.1389} = 0.36$$

Answer: since 0.64 > 0.36, the tuple is classified as **Result = PASS**.

Variant with only two attributes, X = (Confident = Yes, Sick = No):

$$\text{Pass: } \frac{3}{5}\times\frac{2}{3}\times\frac{1}{3} = 0.133 \qquad\qquad \text{Fail: } \frac{2}{5}\times\frac{1}{2}\times\frac{1}{2} = 0.100$$

The answer is again **PASS** (posterior $= \frac{0.133}{0.233} = 0.57$).

Exam tip: if any count is 0, use **Laplace smoothing**, $P(x_i \mid C) = \dfrac{n_{x_i,C} + 1}{n_C + k}$, where k is the number of possible values of that attribute.

N3. Candidate Elimination: find the S and G boundaries. (7 Marks)

Given: the EnjoySport table of 4 examples (attributes Sky, AirTemp, Humidity, Wind, Water, Forecast). It is traced in full in Unit 1, Q4. The step-by-step summary to write in the exam:

Step	Example	S boundary	G boundary
0	-	⟨ø, ø, ø, ø, ø, ø⟩	⟨?, ?, ?, ?, ?, ?⟩
1	+ Sunny, Warm, Normal, Strong, Warm, Same	⟨Sunny, Warm, Normal, Strong, Warm, Same⟩	⟨?, ?, ?, ?, ?, ?⟩
2	+ Sunny, Warm, High, Strong, Warm, Same	⟨Sunny, Warm, ?, Strong, Warm, Same⟩	⟨?, ?, ?, ?, ?, ?⟩
3	– Rainy, Cold, High, Strong, Warm, Change	unchanged	⟨Sunny,?,?,?,?,?⟩, ⟨?,Warm,?,?,?,?⟩, ⟨?,?,?,?,?,Same⟩
4	+ Sunny, Warm, High, Strong, Cool, Change	⟨Sunny, Warm, ?, Strong, ?, ?⟩	⟨Sunny,?,?,?,?,?⟩, ⟨?,Warm,?,?,?,?⟩

Rules to remember

- **Positive example:** generalize S just enough to cover it, and remove from G anything that does not cover it.
- **Negative example:** specialize G just enough to exclude it, keeping only hypotheses that are more general than S. Remove from S anything that covers it.

Answer: S = ⟨Sunny, Warm, ?, Strong, ?, ?⟩ and G = {⟨Sunny, ?, ?, ?, ?, ?⟩, ⟨?, Warm, ?, ?, ?, ?⟩}

N4. K-Means Clustering: perform the iterations for 2-D points with K = 2. (7 Marks)

Given: six points (Height, Weight). K = 2, with initial centroids C1 = P1 = (185, 72) and C2 = P2 = (170, 56).

Point	P1	P2	P3	P4	P5	P6
(x, y)	(185, 72)	(170, 56)	(168, 60)	(179, 68)	(182, 72)	(188, 77)

Formula used (Euclidean distance)

$$d((x_1, y_1), (x_2, y_2)) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Iteration 1: assign each point to the nearest centroid

Point	Distance to C1 (185, 72)	Distance to C2 (170, 56)	Assigned
P1 (185, 72)	0	$\sqrt{15^2 + 16^2} = 21.93$	C1
P2 (170, 56)	21.93	0	C2
P3 (168, 60)	$\sqrt{17^2 + 12^2} = \sqrt{433} = 20.81$	$\sqrt{2^2 + 4^2} = \sqrt{20} = 4.47$	**C2**
P4 (179, 68)	$\sqrt{6^2 + 4^2} = \sqrt{52} = 7.21$	$\sqrt{9^2 + 12^2} = 15.00$	**C1**
P5 (182, 72)	$\sqrt{3^2 + 0^2} = 3.00$	$\sqrt{12^2 + 16^2} = 20.00$	**C1**
P6 (188, 77)	$\sqrt{3^2 + 5^2} = \sqrt{34} = 5.83$	$\sqrt{18^2 + 21^2} = \sqrt{765} = 27.66$	**C1**

- Cluster 1 = {P1, P4, P5, P6} and Cluster 2 = {P2, P3}.

Recompute the centroids (mean of the points in each cluster)

$$C_1 = \left(\frac{185 + 179 + 182 + 188}{4}, \frac{72 + 68 + 72 + 77}{4}\right) = (183.5,\ 72.25)$$

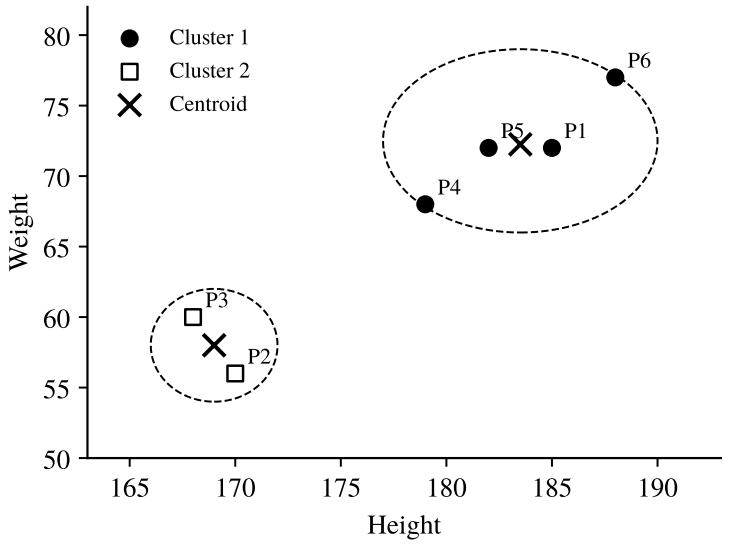
$$C_2 = \left(\frac{170 + 168}{2}, \frac{56 + 60}{2}\right) = (169,\ 58)$$

Iteration 2: re-assign with the new centroids

Point	Distance to C1 (183.5, 72.25)	Distance to C2 (169, 58)	Assigned
P1 (185, 72)	$\sqrt{1.5^2 + 0.25^2} = 1.52$	$\sqrt{16^2 + 14^2} = 21.26$	C1
P2 (170, 56)	$\sqrt{13.5^2 + 16.25^2} = 21.13$	$\sqrt{1^2 + 2^2} = 2.24$	C2
P3 (168, 60)	$\sqrt{15.5^2 + 12.25^2} = 19.76$	$\sqrt{1^2 + 2^2} = 2.24$	C2
P4 (179, 68)	$\sqrt{4.5^2 + 4.25^2} = 6.19$	$\sqrt{10^2 + 10^2} = 14.14$	C1
P5 (182, 72)	$\sqrt{1.5^2 + 0.25^2} = 1.52$	$\sqrt{13^2 + 14^2} = 19.10$	C1
P6 (188, 77)	$\sqrt{4.5^2 + 4.75^2} = 6.54$	$\sqrt{19^2 + 19^2} = 26.87$	C1

Answer: no point changed its cluster, so the algorithm has **converged**.

- **Cluster 1 = {P1, P4, P5, P6}**, centroid **(183.5, 72.25)**
- **Cluster 2 = {P2, P3}**, centroid **(169, 58)**

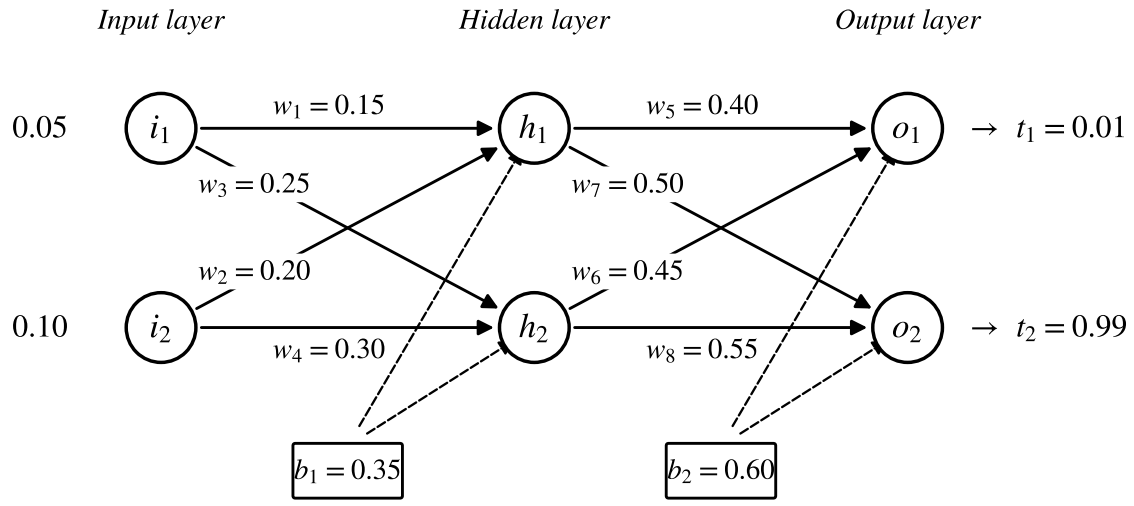


Final clusters after convergence (x = centroid)

Note: some textbooks use the **incremental** version, which updates the centroid right after each point is assigned. Both versions give the same final clusters here. State which version you are using.

N5. Backpropagation: one forward pass and one backward pass with η = 0.5. (7 Marks)

Given network (2 inputs, 2 hidden units, 2 outputs, sigmoid activation everywhere)



Network with initial weights, biases and targets

- Inputs: $i_1 = 0.05,\ i_2 = 0.10$. Targets: $t_1 = 0.01,\ t_2 = 0.99$
- Biases: $b_1 = 0.35$ (hidden), $b_2 = 0.60$ (output). Learning rate $\eta = 0.5$

Formulas used

$$net_j = \sum_i w_{ji} x_i + b, \qquad out_j = \sigma\left(net_j\right) = \frac{1}{1 + e^{-net_j}}, \qquad E = \frac{1}{2}\sum_k \left(t_k - o_k\right)^2$$

$$\delta_k = \left(t_k - o_k\right) o_k \left(1 - o_k\right) \quad \text{(output)}, \qquad \delta_h = o_h \left(1 - o_h\right) \sum_k w_{kh}\, \delta_k \quad \text{(hidden)}, \qquad w_{\text{new}} = w + \eta\, \delta_j\, x_j$$

Part A: Forward pass

Hidden layer

$$net_{h_1} = w_1 i_1 + w_2 i_2 + b_1 = 0.15\left(0.05\right) + 0.20\left(0.10\right) + 0.35 = 0.3775$$

$$out_{h_1} = \frac{1}{1 + e^{-0.3775}} = 0.5933$$

$$net_{h_2} = 0.25\left(0.05\right) + 0.30\left(0.10\right) + 0.35 = 0.3925 \quad \Rightarrow \quad out_{h_2} = 0.5969$$

Output layer

$$net_{o_1} = w_5\, out_{h_1} + w_6\, out_{h_2} + b_2 = 0.40\left(0.5933\right) + 0.45\left(0.5969\right) + 0.60 = 1.1059$$

$$out_{o_1} = \sigma\left(1.1059\right) = 0.7514$$

$$net_{o_2} = 0.50\left(0.5933\right) + 0.55\left(0.5969\right) + 0.60 = 1.2249 \quad \Rightarrow \quad out_{o_2} = 0.7729$$

Error

$$E = \frac{1}{2}\left(0.01 - 0.7514\right)^2 + \frac{1}{2}\left(0.99 - 0.7729\right)^2 = 0.2748 + 0.0236 = 0.2984$$

Part B: Backward pass (output layer)

$$\delta_{o_1} = \left(t_1 - out_{o_1}\right) out_{o_1} \left(1 - out_{o_1}\right) = \left(0.01 - 0.7514\right)\left(0.7514\right)\left(0.2486\right) = -0.1385$$

$$\delta_{o_2} = (0.99 - 0.7729)(0.7729)(0.2271) = +0.0381$$

Weight updates ($\Delta w = \eta\, \delta\, out_h$)

$$w_5^{\text{new}} = 0.40 + 0.5(-0.1385)(0.5933) = 0.3589$$

$$w_6^{\text{new}} = 0.45 + 0.5(-0.1385)(0.5969) = 0.4087$$

$$w_7^{\text{new}} = 0.50 + 0.5(0.0381)(0.5933) = 0.5113$$

$$w_8^{\text{new}} = 0.55 + 0.5(0.0381)(0.5969) = 0.5614$$

Part C: Backward pass (hidden layer). Use the **old** values of w5 to w8.

$$\delta_{h_1} = out_{h_1}\left(1 - out_{h_1}\right)[w_5\,\delta_{o_1} + w_7\,\delta_{o_2}] = 0.2413\,[0.40(-0.1385) + 0.50(0.0381)] = 0.2413(-0.036$$

$$\delta_{h_2} = 0.2406\,[0.45(-0.1385) + 0.55(0.0381)] = 0.2406(-0.0414) = -0.00995$$

Weight updates ($\Delta w = \eta\, \delta_h\, i$)

$$w_1^{\text{new}} = 0.15 + 0.5(-0.00877)(0.05) = 0.14978$$

$$w_2^{\text{new}} = 0.20 + 0.5(-0.00877)(0.10) = 0.19956$$

$$w_3^{\text{new}} = 0.25 + 0.5(-0.00995)(0.05) = 0.24975$$

$$w_4^{\text{new}} = 0.30 + 0.5(-0.00995)(0.10) = 0.29950$$

Answer (summary table)

Weight	w1	w2	w3	w4	w5	w6	w7	w8
Old	0.15	0.20	0.25	0.30	0.40	0.45	0.50	0.55
New	0.14978	0.19956	0.24975	0.29950	0.3589	0.4087	0.5113	0.5614

- Total error before the update: **E = 0.2984**. Running the forward pass again with the new weights gives a smaller error, which shows that learning is happening.
- *Exam tip:* update the hidden-layer weights using the **old** output weights, and always write the δ formula before substituting.
